Machine Learning manuscript No.
(will be inserted by the editor)

The Kernel Kalman Rule

Efficient Nonparametric Inference by Recursive Least-Squares and Subspace Projections

Gregor H. W. Gebhardt · Andras Kupcsik · Gerhard Neumann

Received: date / Accepted: date

Abstract Enabling robots to act in unstructured and unknown environments requires versatile state estimation techniques. While traditional state estimation methods require known models and make strong assumptions about the dynamics, such versatile techniques should be able to deal with high dimensional observations and non-linear, unknown system dynamics. The recent framework for nonparametric inference (Song et al 2013) allows to perform inference on arbitrary probability distributions. High-dimensional embeddings of distributions into reproducing kernel Hilbert spaces (RKHS) are manipulated by kernelized inference rules, most prominently the kernel Bayes' rule (KBR). However, the computational demands of the KBR do not scale with the number of samples. In this paper, we present two techniques to increase the computational efficiency of non-parametric inference. First, the kernel Kalman rule (KKR) is presented as an alternative to the KBR. The derivation of the KKR follows the clear objective of recursive least squares. Based on the KKR we present the kernel Kalman filter (KKF) that updates an embedding of the belief state and learns the system and observation models from data. We further derive the kernel forward backward smoother (KFBS) based on a forward and backward KKF and a smoothing update in Hilbert space. Second, we present the subspace conditional embedding operator as a sparsification technique that still leverages from the full data set. We apply this sparsification to the KKR and derive the corresponding sparse KKF and KFBS algorithms. We show on nonlinear state estimation tasks that our approaches provide a significantly improved estimation accuracy while the computational demands are considerably decreased.

Gregor H.W. Gebhardt
Technische Universität Darmstadt, Computational Learning for Autonomous Systems
Hochschulstr. 10, 64285 Darmstadt, Germany
E-mail: gregor@robot-learning.de Tel.: +49-6151-16-25371
0000-0002-8575-069X

Andras Kupcsik
Idiap Research Institute, Centre du Parc
Rue Marconi 19, PO Box 592, 1920 Martigny, Switzerland
E-mail: andras.kupcsik@idiap.ch

Gerhard Neumann
University of Lincoln, Lincoln Centre for Autonomous Systems
Brayford Pool, LN6 7TS, Lincoln, UK
E-mail: geri@robot-learning.de

Keywords nonparametric inference · kernel methods · state estimation · model learning

1 Introduction

The ability to reason about past, current and future states in continuous, partially observable stochastic processes is a fundamental stepstone towards fully autonomos and intelligent systems. Such models are required in many applications as for example state estimation in case of incomplete sensory data, smoothing noisy data from mediocre sensors, or predicting future states from past and current observations.

Traditional state estimation techniques usually require analytical models of the underlying system, are often limited to a set of models with a special structure, and require knowledge about the moments of the stochstic processes. When assuming linear Gaussian models with known mean and covariance for instance, the Kalman filter (Kalman 1960) yields the optimal solution. However, the requirement of linear Gaussian models with known statistics imposes a strong limitation to the applicability of this method. For more complex processes, approximate solutions have to be used instead. Examples are the extended Kalman filter (McElhoe 1966; Smith et al 1962) or the unscented Kalman filter (Julier and Uhlmann 1997; Wan and Van Der Merwe 2000). These solution inherit the Gaussian representation of the belief state to which they apply the non-linear system dynamics. However, the Gaussian distribution with its unimodal nature is a strong assumption about the belief state which leads to poor results for systems that yield a more complex distribution over possible states. Moreover, both the Kalman filter but also it's extensions to non-linear systems, require that the dynamics of the systems are known as analytical models. Yet, these analytical models are often hard to obtain or make simplifying assumptions about the system.

The recently introduced framework for nonparametric inference (Song et al 2013; Fukumizu et al 2013) alleviate the problems of traditional state estimation methods for nonlinear systems. The basic idea of these methods is to embed the probability distributions into reproducing kernel Hilbert spaces (RKHS). These embeddings allow the representation of arbitrary probability distributions using empirical estimators. With the kernelized versions of the sum rule, the chain rule, and the Bayes' rule, inference on the embedded distribution can then be performed efficiently and entirely in the RKHS. Additionally, Song et al (2013) use the kernel sum rule and the kernel Bayes' rule to construct the kernel Bayes' filter (KBF). The KBF learns the transition and observation models from observed samples and can be applied to nonlinear systems with high-dimensional observations. However, the computational complexity of the KBR update scales poorly with the number of samples such that hyper-parameter optimization becomes prohibitively expensive. Moreover, the KBR requires mathematical tricks that may cause numerical instabilities and also render the objective which is optimized by the KBR unclear.

We propose two additions to the framework for nonparametric inference to overcome the limitations named above. First, we introduce the *subspace conditional embedding operator*. In contrast to the conditional embedding operator (Song et al 2009), this operator allows to estimate its empirical estimator with a much larger data set while maintaining computational efficiency. We further apply this subspace conditional embedding operator to the kernel sum rule, kernel chain rule and kernel Bayes rule to derive their subspace versions. We presented these results in a workshop paper at the large-scale kernel learning workshop at ICML 2015 (Gebhardt et al 2015).

Furthermore, we present the *kernel Kalman rule* (KKR) as alternative to the kernel Bayes' rule. Our derivations closely follow the derivations of the innovation update used

in the Kalman filter and are based on a recursive least squares minimization objective in a reproducing kernel Hilbert space (RKHS). We show that our update of the mean embedding is unbiased and has minimum variance. While the update equations are formulated in a potentially infinite dimensional RKHS, we derive, through the application of the kernel trick and by virtue of the representer theorem, an algorithm using only products and inversions of finite kernel matrices. We employ the kernel Kalman rule together with the kernel sum rule for filtering, which results in the *kernel Kalman filter* (KKF). In contrast to filtering techniques that rely on the KBR, the KKF allows to precompute expensive matrix inversions which significantly reduces the computational complexity and which also allows us to apply hyper-parameter optimization for the KKF. This work has been presented at AAAI 2017 (Gebhardt et al 2017).

In addition to the KKF, we introduce the *kernel forward backward smoother* (KFBS) which computes the embedding of the belief state given all available observations from the past and the future. The kernel forward backward smoother combines the belief state embeddings of a forward pass and a backward pass into smoothed embeddings using Hilbert space operations. Both, the forward and the backward pass are realized by a KKF, where the backward KKF operates backwards in time starting at the last observation. To scale gracefully with larger data sets, we rederive the KKR, the KKF and the KFBS with the subspace conditional operator (Gebhardt et al 2015).

We compare our approach to different versions of the KBR and demonstrate its improved estimation accuracy and computational efficiency. Furthermore, we evaluate the KKR on a simulated 4-link pendulum task, on a human motion capture data set (Wojtusch and von Stryk 2015) and on data from a table-tennis setup (Gomez-Gonzalez et al 2016).

1.1 Related Work

To the best of our knowledge the kernel Bayes' rule exists in three different versions. It was first introduced in its original version by Fukumizu et al (2013). Here, the KBR is derived, similar to the conditional operator, using prior modified covariance operators. These prior-modified covariance operators are approximated by weighting the feature mappings with the weights of the embedding of the prior distribution. Since these weights are potentially negative, the covariance operator might become indefinite, and thus, rendering its inversion impossible. To overcome this drawback, the authors have to apply a form of the Tikhonov regularization that significantly decreases accuracy and increases the computational costs. A second version of the KBR was introduced by Song et al (2013) in which they use a different approach to approximate the prior-modified covariance operator. In the experiments conducted for this paper, this second version often leads to more stable algorithms than the first version. Boots et al (2013) introduced a third version of the KBR where they apply only the simple form of the Tikhonov regularization. However, this rule requires the inversion of a matrix that is often indefinite, and therefore, high regularization constants are required, which again degrades the performance. In our experiments, we refer to these different versions with KBR(b) for the first, KBR(a) for the second (both adapted from the literature), and KBR(c) for the third version. Song et al (2013) propose in their framework for nonparametric inference to combine the KBR with the kernel sum rule to obtain the kernel Bayes filter (KBF). Our kernel Kalman filter is closely related to this, as we simply replace the KBR with the KKR. We compare to the KBF in our experiments. Nishiyama et al (2016) recently proposed the nonparametric kernel Bayes smoother. This approach builds on top of the kernel Bayes filter, which is used

to compute the estimates of a normal forward pass. The smoothing update is then obtained by propagating the embeddings backwards in time without performing a second filtering pass.

For filtering tasks with known linear system equations and Gaussian noise, the Kalman filter (KF) yields the solution that minimizes the squared error of the estimate to the true state. Two widely known and applied approaches to extend the Kalman filter to non-linear systems are the *extended Kalman filter* (EKF) (McElhoe 1966; Smith et al 1962) and the *unscented Kalman filter* (UKF) (Wan and Van Der Merwe 2000; Julier and Uhlmann 1997). Both, the EKF and the UKF, assume that the non-linear system dynamics are known and use them to update the prediction mean. Yet, updating the prediction covariance is not straightforward. In the EKF the system dynamics are linearized at the current estimate of the state, and in the UKF the covariance is updated by applying the system dynamics to a set of sample-points (sigma points). While these approximations make the computations tractable, they can significantly reduce the quality of the state estimation, in particular for high-dimensional systems.

Hsu et al (2012) recently proposed an algorithm for learning Hidden Markov Models (HMMs) by exploiting the spectral properties of observable measures to derive an observable representation of the HMM (Jaeger 2000). An RKHS embedded version thereof was presented in (Song et al 2010). While this method is applicable for continuous state spaces, it still assumes a finite number of discrete hidden states.

Other closely related algorithms to our approach are the kernelized version of the Kalman filter and Kalman smoother by Ralaivola and d'Alche Buc (2005) and the kernel Kalman filter based on the conditional embedding operator (KKF-CEO) by Zhu et al (2014). The former approach formulates the Kalman filter in a sub-space of the infinite feature space that is defined by the kernels. Hence, this approach does not fully leverage the kernel idea of using an infinite feature space. In contrast, the KKF-CEO approach embeds the belief state also in an RKHS. However, they require that the observation is a noisy version of the full state of the system, and thus, they cannot handle partial observations. Moreover, they also deviate from the standard derivation of the Kalman filter, which, as our experiments show, decreases the estimation accuracy. The full observability assumption is needed in order to implement a simplified version of the innovation update of the Kalman filter in the RKHS. Our algorithm, the KKF does not suffer from this restriction. It also provides update equations that are much closer to the original Kalman filter and outperforms the KKF-CEO algorithm as shown in our experiments. Another approach to state estimation is presented in (Kawahara et al 2007). In this paper the authors propose to estimate low-dimensional state vectors based on kernel canonical correlation analysis and then regress a linear transition model of the estimated state vectors and the nonlinear features of the input.

Learning predictors in the space of predictive state representations to perform filtering has been proposed in (Sun et al 2016b) and later extended to smoothing in (Sun et al 2016a). They introduce predictive state inference machines (PSIM) which are (nonlinear) regressors on predictive states learned from data to perform filtering. With the smoothing machine (SMACH) they extend this concept for smoothing.

2 Preliminaries

Our work is based on the recent formulations of embedding distributions into reproducing kernel Hilbert spaces (Smola et al 2007; Song et al 2013). These embeddings allow to represent arbitrary probability distributions non-parametrically by a potentially infinite dimensional feature vector. Through the application of derived operators (Song et al 2009;

Fukumizu et al 2011) it is furthermore possible to apply inference rules entirely in the Hilbert space. In the first part of this section, we want to give the reader an introduction into this technology and define the notation we will use throughout this article. We will furthermore show a modification of this framework which allows to learn the embeddings and inference rules with much larger datasets while maintaining a low computational complexity.

One of the main contributions of our paper is a novel method for performing Bayesian updates on a distribution embedded into an RKHS. The derivations of this update rule are based on a least-squares objective and inspired by the derivations of the Kalman filter update, thus we name this method *kernel Kalman rule*. In the second part of this section, we will recapitulate the classical Kalman filter equations and review the derivations of the innovation update based on the least-squares objective.

2.1 Nonparametric Inference with Hilbert Space Embeddings of Distributions

Intuitively, a Hilbert space is an extension of the well known two- or three-dimensional Euclidean vector space to arbitrary many dimensions, specifically including infinite dimensional vector spaces. Such infinite dimensional Hilbert spaces include spaces where the single elements are functions, i.e., infinite dimensional vectors that contain for each element of the domain the corresponding function value in the image. In addition, a Hilbert space has an inner product that allows to measure distances and angles between its elements. For a reproducing kernel Hilbert space $\mathcal{H}_k$, this inner product $\langle \cdot, \cdot \rangle$ is implicitly defined by a reproducing kernel, with $k : \Omega \times \Omega \to \mathbb{R}$ and $k(\boldsymbol{x}, \boldsymbol{y}) =: \langle \boldsymbol{\varphi}(\boldsymbol{x}), \boldsymbol{\varphi}(\boldsymbol{y}) \rangle$, where $\boldsymbol{\varphi}(\boldsymbol{x})$ is a feature mapping into a possibly infinite dimensional space, intrinsic to the kernel function. For example the Gaussian kernel computes the inner product of the feature mappings of its inputs where the feature mappings itself cannot be written down explicitly as they are into an infinite dimensional space. Due to the reproducing property of the kernel, all elements f of the RKHS can be reproduced by k in the sense that the outcome $f(x)$ of the function for a specific value x can obtained by an evaluation of the kernel function (Aronszajn 1950), i.e., $f(\boldsymbol{y}) = \langle f, \boldsymbol{\varphi}(\boldsymbol{y}) \rangle$ for any $f \in \mathcal{H}_k$.

In a practical setting, we want to embed probability distributions in an RKHS spanned by samples $\mathcal{D} = \left[(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n) \right]$. Based on the representer theorem (Schölkopf et al 2001) and the reproducing property, the elements f of an RKHS $\mathcal{H}_k$ can then be written as

$$f(\cdot) = \sum_{i=1}^{n} \boldsymbol{\alpha}_i k(\boldsymbol{x}_i, \cdot) = \sum_{i=1}^{n} \boldsymbol{\alpha}_i \langle \boldsymbol{\varphi}(\boldsymbol{x}_i), \boldsymbol{\varphi}(\cdot) \rangle = \boldsymbol{\alpha}^\mathsf{T} \boldsymbol{\Upsilon}_{\boldsymbol{x}}^\mathsf{T} \boldsymbol{\varphi}(\cdot), \tag{1}$$

with the weights $\alpha_i \in \mathbb{R}$ and where we denote the feature matrix of samples $\boldsymbol{x}_i$ by $\boldsymbol{\Upsilon}_{\boldsymbol{x}} = [\boldsymbol{\varphi}(\boldsymbol{x}_1), \ldots, \boldsymbol{\varphi}(\boldsymbol{x}_n)]$. In the following paragraphs we will show how probability distributions can be represented as an embedding in such a reproducing kernel Hilbert space and how the operators for performing inference in the RKHS can be derived.

2.1.1 Embeddings of Marginal and Joint Distributions

The embedding of a marginal density $P(X)$ over the random variable X is defined as the expected feature mapping $\boldsymbol{\mu}_X := \mathbb{E}_X [\boldsymbol{\phi}(X)]$, also called the *mean map* (Smola et al 2007). Using a finite set of samples from $P(X)$, the mean map can be estimated as

$$\hat{\boldsymbol{\mu}}_X = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\varphi}(\boldsymbol{x}_i) = \frac{1}{n} \boldsymbol{\Upsilon}_{\boldsymbol{x}}^\mathsf{T} \mathbf{1}_n, \tag{2}$$

where Υ_x is the feature matrix of the samples from the distribution and $\mathbf{1}_n \in \mathbb{R}^n$ is an n dimensional vector of ones. Because of the reproducing property of the kernel function, computing the expectation of a function which is an element of the same RKHS resolves to simple matrix operations. On the other hand, the probability of a single outcome or higher order statistics of the distributions are not straight forward to obtain.

Alternatively, a distribution can be embedded in a tensor product RKHS $\mathcal{H}_k \times \mathcal{H}_k$ as the expected tensor product of the feature mappings (Smola et al 2007)

$$C_{XX} := \mathbb{E}_{XX}\left[\boldsymbol{\varphi}(X) \otimes \boldsymbol{\varphi}(X)\right] - \boldsymbol{\mu}_X \otimes \boldsymbol{\mu}_X, \tag{3}$$

where we use $\otimes$ to denote the tensor product (or outer product) of two vectors. This embedding is also called the *centered covariance operator*. The finite sample estimator is given by

$$\hat{C}_{XX} = \frac{1}{m}\sum_{i=1}^{m} \boldsymbol{\varphi}(\boldsymbol{x}_i) \otimes \boldsymbol{\varphi}(\boldsymbol{x}_i) - \hat{\boldsymbol{\mu}}_X \otimes \hat{\boldsymbol{\mu}}_X. \tag{4}$$

Similarly, we can define the *uncentered cross-covariance operator* for a joint distribution $p(X, Y)$ of two variables X and Y as $\hat{C}_{XY} = \frac{1}{m}\sum_{i=1}^{m} \boldsymbol{\varphi}(x_i) \otimes \boldsymbol{\phi}(y_i)$. Based on these two definitions for Hilbert space embeddings of probability distributions and with the conditional embedding operator discussed in the next section, all the rules of the framework for non-parametric inference (Song et al 2013) can be derived.

2.1.2 The Conditional Embedding Operator

The embedding of a conditional distribution $P(Y|X)$ is not like the mean map a single element of the RKHS, but rather a family of embeddings where there is one embedding for each realization of the conditioning variable X. To obtain the conditional distribution for a specific value $X = \boldsymbol{x}_*$, Song et al (2009) defined the *conditional embedding operator* $C_{Y|X}$ which, if applied to the feature mapping of $\boldsymbol{x}$ returns the embedding of $P(Y|X = \boldsymbol{x}_*)$

$$\boldsymbol{\mu}_{Y|\boldsymbol{x}} := \mathbb{E}_{Y|\boldsymbol{x}}\left[\boldsymbol{\phi}(Y)\right] = C_{Y|X}\boldsymbol{\varphi}(\boldsymbol{x}). \tag{5}$$

Given a finite set of samples $\mathcal{D} = \left[(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n)\right]$ from the conditional distribution $P(Y|X)$, the conditional embedding operator can be derived from a least-squares objective (Grünewälder et al 2012) as

$$\hat{C}_{Y|X} = \boldsymbol{\Phi}_y(\boldsymbol{K} + \lambda \boldsymbol{I}_n)^{-1}\Upsilon_x^\intercal, \tag{6}$$

with the feature matrices $\boldsymbol{\Phi}_y := [\boldsymbol{\phi}(\boldsymbol{y}_1), \ldots, \boldsymbol{\phi}(\boldsymbol{y}_n)]$ and $\Upsilon_x := [\boldsymbol{\varphi}(\boldsymbol{x}_1), \ldots, \boldsymbol{\varphi}(\boldsymbol{x}_n)]$, the Gram matrix $\boldsymbol{K} = \Upsilon_x^\intercal\Upsilon_x \in \mathbb{R}^{n\times n}$, the regularization parameter λ, and the identity matrix $\boldsymbol{I}_n \in \mathbb{R}^{n\times n}$. With the feature mapping of the realization $\boldsymbol{x}_*$ this results in

$$\boldsymbol{\mu}_{Y|\boldsymbol{x}_*} = \hat{C}_{Y|X}\boldsymbol{\varphi}(\boldsymbol{x}_*) = \boldsymbol{\Phi}_y(\boldsymbol{K} + \lambda \boldsymbol{I}_n)^{-1}\Upsilon_x^\intercal\boldsymbol{\varphi}(\boldsymbol{x}_*) = \boldsymbol{\Phi}_y(\boldsymbol{K} + \lambda \boldsymbol{I}_n)^{-1}\boldsymbol{k}_{\boldsymbol{x}_*}, \tag{7}$$

where $\boldsymbol{k}_{\boldsymbol{x}_*}$ is the kernel vector of the samples $[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]$ and the realization $\boldsymbol{x}_*$. As the kernel matrices in the inverse and the kernel vector of the realization are finite, the embedding of the conditional distribution can be represented as a weighted sum of feature mappings

$$\boldsymbol{\mu}_{Y|\boldsymbol{x}_*} = \boldsymbol{\Phi}_y\boldsymbol{\alpha} = \sum_{i=1}^{n} \alpha_i \boldsymbol{\phi}(\boldsymbol{y}_i), \tag{8}$$

with the feature mappings of the samples $[\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n]$ and the finite weight vector $\boldsymbol{\alpha} \in \mathbb{R}^n$.

2.1.3 The Kernel Sum Rule

The embedding of $Q(Y) = \sum_X P(Y|X)\pi(X)$ can be obtained from the *kernel sum rule* (Song et al 2013). To that end, the conditional operator is applied to the embedding $\boldsymbol{\mu}_X^\pi = \boldsymbol{\Upsilon}_x \boldsymbol{\alpha}_\pi$ of the prior distribution $\pi(X)$,

$$\hat{\mu}_Y^\pi = \hat{C}_{Y|X}\mu_X^\pi = \boldsymbol{\Phi}_y(\boldsymbol{K} + \lambda \mathbf{I}_m)^{-1}\boldsymbol{K}\boldsymbol{\alpha}_\pi. \tag{9}$$

Here, we can see again that the result can be represented as a weighted sum over feature mappings. In order to obtain the distribution $Q(Y)$ as a covariance operator instead of a mean map, Song et al (2013) also proposed the kernel sum rule for tensor product features which yields the prior modified covariance operator C_{YY}^π as

$$C_{YY}^\pi = C_{(YY)|X}\mu_X^\pi \tag{10}$$

$$C_{YY}^\pi = \boldsymbol{\Phi}_y \mathrm{diag}((\boldsymbol{K} + \lambda \boldsymbol{I}_m)^{-1}\boldsymbol{K}\boldsymbol{\alpha})\boldsymbol{\Phi}_y^\intercal, \tag{11}$$

where $C_{(YY)|X}$ is the conditional operator for tensor product features.

2.1.4 The Kernel Chain Rule

The *kernel chain rule* (Song et al 2013) yields an embedding of the joint distribution $Q(X,Y) = P(Y|X)\pi(X)$ as a prior modified covariance operator. There are two versions of the kernel chain rule. Both apply the conditional embedding operator of $P(Y|X)$ to an embedding of the prior distribution $\pi(X)$. In the first version the conditional operator is applied to a covariance embedding of the prior distribution. But, this covariance operator is not estimated directly from samples but approximated from the weight vector $\boldsymbol{\alpha}_\pi$ of the mean map $\boldsymbol{\mu}_X^\pi = \boldsymbol{\Upsilon}_x \boldsymbol{\alpha}_\pi$ as $\hat{C}_{XX}^\pi = \boldsymbol{\Upsilon}_x \mathrm{diag}(\boldsymbol{\alpha})\boldsymbol{\Upsilon}_x^\intercal$. This yields Version (a) of the kernel chain rule as

$$\hat{C}_{YX}^\pi = C_{X|Y}\hat{C}_{XX}^\pi = \boldsymbol{\Phi}_y(\boldsymbol{K} + \lambda \mathbf{I}_m)^{-1}\boldsymbol{K}\mathrm{diag}(\boldsymbol{\alpha})\boldsymbol{\Upsilon}_x^\intercal. \tag{12}$$

Version (b) of the kernel chain rule first computes the mean map μ_Y^π conditioned on the prior distribution $\pi(X)$ by applying the conditional embedding operator to the mean map μ_X^π. Afterwards, the prior-modified covariance operator of the joint distribution is constructed from the resulting weight vector which results in

$$\hat{C}_{YX}^\pi = \boldsymbol{\Phi}_y \mathrm{diag}((\boldsymbol{K} + \lambda \mathbf{I}_m)^{-1}\boldsymbol{K}\boldsymbol{\alpha})\boldsymbol{\Upsilon}_x^\intercal. \tag{13}$$

Both versions of the kernel chain rule have been used to derive different versions of the kernel Bayes' rule as we will depict below.

2.1.5 The Kernel Bayes' Rule

Given the embedding of a prior distribution $\pi(X)$ and the feature mapping of an observation $\boldsymbol{\phi}(\boldsymbol{y})$, the *kernel Bayes' rule* (KBR) infers the mean embedding of the posterior distribution $Q_\pi(X|Y = \boldsymbol{y})$. The idea is to construct a prior-modified conditional embedding operator that yields the mean map of the posterior if applied to the feature mapping of the observation (Fukumizu et al 2013)

$$\boldsymbol{\mu}_{X|\boldsymbol{y}}^\pi = C_{X|Y}^\pi \boldsymbol{\phi}(\boldsymbol{y}). \tag{14}$$

This prior-modified conditional operator is constructed from two prior-modified covariance operators C^{π}_{YY} and C^{π}_{YX} obtained from the kernel sum and the kernel chain rule, respectively, using the relation

$$C^{\pi}_{X|Y} = C^{\pi}_{XY} \left(C^{\pi}_{YY}\right)^{-1}. \tag{15}$$

In the first version, which we denote by KBR(b) following the notation of Song et al (2013), Fukumizu et al (2013) derived the kernel Bayes' rule using the tensor product conditional operator in the kernel chain rule (c.f. Equation 13) and arrived at

$$\hat{\boldsymbol{\mu}}^{\pi}_{X|y} = \Upsilon_x \boldsymbol{D}\boldsymbol{G} \left((\boldsymbol{D}\boldsymbol{G})^2 + \kappa \boldsymbol{I}_m\right)^{-1} \boldsymbol{D}\boldsymbol{g}_y, \tag{16}$$

with the diagonal $\boldsymbol{D} := \mathrm{diag}((\boldsymbol{K}_{xx} + \lambda \boldsymbol{I}_m)^{-1}\boldsymbol{K}\boldsymbol{\alpha})$, the gram matrix $\boldsymbol{G} = \boldsymbol{\Phi}_y^{\intercal}\boldsymbol{\Phi}_y$, the kernel vector $\boldsymbol{g}_y$ and κ as regularization parameter. Song et al (2013) derived the KBR using the first formulation of the kernel chain rule shown in Equation 3.2 which results in

$$\hat{\boldsymbol{\mu}}^{\pi}_{X|y} = \Upsilon_x \boldsymbol{\Lambda}^{\intercal} \left((\boldsymbol{D}\boldsymbol{G})^2 + \kappa \boldsymbol{I}_m\right)^{-1} \boldsymbol{G}\boldsymbol{D}\boldsymbol{g}_y, \tag{17}$$

with $\boldsymbol{\Lambda} := (\boldsymbol{K} + \lambda \boldsymbol{I}_m)^{-1}\boldsymbol{K}\mathrm{diag}(\boldsymbol{\alpha})$. This second version of the kernel Bayes' rule is denoted by KBR(a). As the matrix $\boldsymbol{D}\boldsymbol{G}$ is typically not invertible, both of these versions of the KBR use a form of the Tikhonov regularization in which the matrix in the inverse is squared. Boots et al (2013) use a third form of the KBR which is derived analogously to the first version but does not use the squared form of Tikhonov regularization, i.e.,

$$\hat{\boldsymbol{\mu}}^{\pi}_{X|y} = \Upsilon_x \left(\boldsymbol{D}\boldsymbol{G} + \kappa \boldsymbol{I}_m\right)^{-1} \boldsymbol{D}\boldsymbol{g}_y. \tag{18}$$

Since the product $\boldsymbol{D}\boldsymbol{G}$ is often not positive definite, a strong regularization parameter is required to make the matrix invertible. We denote this third version of the kernel Bayes' rule consequently by KBR(c).

All three versions of the kernel Bayes' rule presented above have drawbacks. First, due to the approximation of the prior modified covariance operators, these operators are not guaranteed to be positive definite and, thus, their inversion requires either a harsh form of the Tikhonov regularization or a strong regularization factor and are often still numerically instable. Furthermore, the inverse is dependent on the embedding of the prior distribution and, hence, needs to be recomputed for every Bayesian update of the mean map. This recomputation significantly increases the computational costs, for example, if we want to apply hyper-parameter optimization techniques.

2.2 The Kalman Filter

The Kalman filter is a well known technique for state estimation, prediction, and smoothing in environments with linear system dynamics that are subject to zero-mean Gaussian noise with known covariances (Kalman 1960). The system equations can be formulated as

$$\boldsymbol{x}_{t+1} = \boldsymbol{F}\boldsymbol{x}_t + \boldsymbol{v}_t, \quad \boldsymbol{y}_t = \boldsymbol{H}\boldsymbol{x}_t + \boldsymbol{w}_t, \tag{19}$$

where $\boldsymbol{x}_t$ is the latent state of the system at time t and $\boldsymbol{y}_t$ is the corresponding observation. The linear Gaussian model is defined by the system matrix $\boldsymbol{F}$, the observation matrix $\boldsymbol{H}$, and noise vectors $\boldsymbol{v}_t$ and $\boldsymbol{w}_t$ which are sampled from $\mathcal{N}(\boldsymbol{0}, \boldsymbol{P})$ and $\mathcal{N}(\boldsymbol{0}, \boldsymbol{R})$, respectively.

From the assumption of Gaussian transition noise and Gaussian observation noise, it follows naturally that the belief state over the latent state $\boldsymbol{x}_t$ is as well a Gaussian distribution with mean $\boldsymbol{\mu}_{\boldsymbol{x},t}$ and covariance $\boldsymbol{\Sigma}_{\boldsymbol{x},t}$. The Kalman filter applies iteratively two update procedures to the belief state to which we will refer to as *prediction* and *innovation* update. During the prediction update the Kalman filter propagates the belief state in time by applying the transition model, i.e.,

$$\boldsymbol{\mu}^-_{\boldsymbol{x},t+1} = \boldsymbol{F}\boldsymbol{\mu}^+_{\boldsymbol{x},t}, \quad \boldsymbol{\Sigma}^-_{\boldsymbol{x},t+1} = \boldsymbol{F}\boldsymbol{\Sigma}^+_{\boldsymbol{x},t}\boldsymbol{F}^\intercal + \boldsymbol{P}. \tag{20}$$

On new observations $\boldsymbol{y}_t$, the innovation update applies Bayes' theorem to the *a-priori* belief state $\{\boldsymbol{\mu}^-_{\boldsymbol{x},t}, \boldsymbol{\Sigma}^-_{\boldsymbol{x},t}\}$ to obtain the *a-posteriori* mean and covariance as

$$\boldsymbol{\mu}^+_{\boldsymbol{x},t} = \boldsymbol{\mu}^-_{\boldsymbol{x},t} + \boldsymbol{Q}_t(\boldsymbol{y}_t - \boldsymbol{H}\boldsymbol{\mu}^-_{\boldsymbol{x},t}), \tag{21}$$

$$\boldsymbol{\Sigma}^+_{\boldsymbol{x},t} = \boldsymbol{\Sigma}^-_{\boldsymbol{x},t} - \boldsymbol{Q}_t\boldsymbol{H}\boldsymbol{\Sigma}^-_{\boldsymbol{x},t}, \tag{22}$$

with the Kalman gain matrix

$$\boldsymbol{Q}_t = \boldsymbol{\Sigma}^-_{\boldsymbol{x},t}\boldsymbol{H}^\intercal(\boldsymbol{H}\boldsymbol{\Sigma}^-_{\boldsymbol{x},t}\boldsymbol{H}^\intercal + \boldsymbol{R})^{-1}.$$

Another approach to derive the Kalman filter equations follows from a least-squares objective between the state estimator a-priori and a-posteriori to the observation Simon (2006). This second approach does not make the explicit assumption that the belief state can be represented as a Gaussian random variable. Rather this representation follows from the objective to minimize the variance of the error between the a-priori and a-posteriori estimators. We will take this second approach as inspiration to derive the kernel Kalman rule in Section 4.

3 Efficient Nonparametric Inference in a Subspace

A general drawback of kernel methods is that the complexities of the algorithms scale poorly with the number of samples in the kernel matrices. As the conditional embedding operator and the kernel inference rules require the inversion of a kernel matrix, the complexity scales cubically with the number of data points. To overcome this drawback, several approaches exist that aim to find a good trade-off between a compact representation and leveraging from a large data set. Examples are the sparse Gaussian processes that use pseudo-inputs (Snelson and Ghahramani 2006; Csató and Opper 2002), or a sparse subset of the data which is selected by maximizing the posterior probability (Smola and Bartlett 2001). Other techniques are based on approximating the kernel matrices using the Nyström method (Williams and Seeger 2000) or random Fourier features (Rahimi and Recht 2007). We approach this problem by proposing the subspace conditional embedding operators (Gebhardt et al 2015). The basic idea is to use only a subset of the available training data as representation for the embeddings but the full data set to learn the conditional operators. In the following sections, we will recapitulate this approach and show how it can be applied to the framework for nonparamteric inference.

Given the feature matrices $\boldsymbol{\Phi}_y := [\boldsymbol{\phi}(\boldsymbol{y}_1), \ldots, \boldsymbol{\phi}(\boldsymbol{y}_n)]$ and $\boldsymbol{\Upsilon}_x := [\boldsymbol{\varphi}(\boldsymbol{x}_1), \ldots, \boldsymbol{\varphi}(\boldsymbol{x}_n)]$, we can define the respective subsets $\boldsymbol{\Psi}_y \subset \boldsymbol{\Phi}_y$ and $\boldsymbol{\Gamma}_x \subset \boldsymbol{\Upsilon}_x$, where $|\boldsymbol{\Psi}_y| = |\boldsymbol{\Gamma}_x| = m \ll n$. We assume that the subsets are representative for the embedded distributions. Similar to the conditional operator discussed in Section 2.1.2, we define the subspace conditional operator $C^S_{Y|X}$ as the mapping from an embedding $\boldsymbol{\varphi}(\boldsymbol{x}) \in \mathcal{H}_X$ to the mean embedding $\mu_{Y|\boldsymbol{x}} \in \mathcal{H}_Y$ of the conditional distribution $P(Y|\boldsymbol{x})$ conditioned on a certain variate $\boldsymbol{x}$. To obtain this subspace conditional operator, we first introduce an auxiliary conditional operator $C^{\text{aux}}_{Y|X}$ which maps

from the subspace projection of the embedding $\mathbf{\Gamma}_x^\intercal \boldsymbol{\varphi}(\boldsymbol{x})$ to the mean map of the conditional distribution, i.e.,

$$\mu_{Y|\boldsymbol{x}} = C_{Y|X}^{\text{aux}} \mathbf{\Gamma}_x^\intercal \boldsymbol{\varphi}(\boldsymbol{x}). \tag{23}$$

We can then derive this auxiliary conditional operator by minimizing the squared error on the full data set

$$C_{Y|X}^{\text{aux}} = \arg\min_C \left\| \mathbf{\Phi}_y - C \mathbf{\Gamma}_x^\intercal \mathbf{\Upsilon}_x \right\|_2 \tag{24}$$

$$= \mathbf{\Phi}_y \mathbf{\Upsilon}_x^\intercal \mathbf{\Gamma}_x \left(\mathbf{\Gamma}_x^\intercal \mathbf{\Upsilon}_x \mathbf{\Upsilon}_x^\intercal \mathbf{\Gamma}_x + \lambda \boldsymbol{I} \right)^{-1}. \tag{25}$$

Substituting this result for the auxiliary conditional operator in Equation 23 gives us the subspace conditional operator as

$$\begin{aligned} C_{Y|X}^{S} &= C_{Y|X}^{\text{aux}} \mathbf{\Gamma}_x^\intercal \\ &= \mathbf{\Phi}_y \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \mathbf{\Gamma}_x^\intercal, \end{aligned} \tag{26}$$

where $\bar{\boldsymbol{K}} = \mathbf{\Upsilon}_x^\intercal \mathbf{\Gamma}_x \in \mathbb{R}^{n \times m}$ is the kernel matrix of the sample feature set $\mathbf{\Upsilon}_x$ and its subset $\mathbf{\Gamma}_x$. Since we assume that $m \ll n$, the inverse in the subspace conditional operator is in $\mathbb{R}^{m \times m}$ and, thus, of a much smaller size than the inverse in the standard conditional operator shown in Equation 6. Additionally, we can make use the feature matrix $\mathbf{\Gamma}_x^\intercal$ on the right hand side of the subspace conditional operator and represent the mean embedding always in the subspace spanned by the features $\mathbf{\Gamma}_x$. This allows to avoid representations and computations in the high-dimensional space spanned by the features of the full sample set. In the following sections we will rederive the kernel sum rule, the kernel chain rule and the kernel Bayes rule based on the subspace conditional embedding operator analogously to Song et al (2013).

3.1 Relation to Other Sparsification Approaches

Many other sparsification techniques for kernel methods exist. The two most important techniques among these are probably the Nyström method (Williams and Seeger 2000; Drineas and Mahoney 2005) and the random Fourier features (Rahimi and Recht 2007). Both methods are closely related to our approach. However, a major difference is that the subspace conditional operator still is an operator in the RKHS defined by the kernel function. The Nyström method as well as the random Fourier features use finite dimensional approximation of the feature function intrinsic to the kernel function and thus the resulting operators are finite matrices that operate on the finite dimensional weight vectors of the mean embedding. See Appendix A.2 on how these sparsification approaches can be applied to the conditional operator.

3.2 The Subspace Kernel Sum Rule

Analogously to Song et al (2013), the subspace kernel sum rule is simply the subspace conditional operator applied to the embedding of a distribution $\pi(X)$, i.e.,

$$\mu_Y^\pi = C_{Y|X}^{S} \mu_X^\pi = \mathbf{\Phi}_y \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \mathbf{\Gamma}_x^\intercal \mathbf{\Upsilon}_x \boldsymbol{\alpha}, \tag{27}$$

where $\mu_X^\pi = \Upsilon_x \boldsymbol{\alpha}$ is the embedding of the prior distribution $\pi(X)$. We construct the subspace kernel sum rule for tensor product features differently than Song et al (2013). Instead of applying the conditional operator to the mean embedding and then approximating the covariance operator with the resulting weights (c.f. Equation 11), we first approximate the covariance operator C_{XX}^π from the weights $\boldsymbol{\alpha}$ of the mean map μ_X^π and then multiply the subspace conditional operator to both sides of it

$$\begin{aligned} C_{YY}^{S,\pi} = C_{Y|X}^S C_{XX}^\pi \left(C_{Y|X}^S\right)^\intercal &= C_{Y|X}^S \Upsilon_x \operatorname{diag}(\boldsymbol{\alpha}) \Upsilon_x^\intercal \left(C_{Y|X}^S\right)^\intercal \\ &= \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \boldsymbol{L}_{\bar{\boldsymbol{K}}} \operatorname{diag}(\boldsymbol{\alpha}) \boldsymbol{L}_{\bar{\boldsymbol{K}}}^\intercal \bar{\boldsymbol{K}}^\intercal \boldsymbol{\Phi}_y^\intercal. \end{aligned} \tag{28}$$

Here, we denote $\boldsymbol{L}_{\bar{\boldsymbol{K}}} := \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I}\right)^{-1} \bar{\boldsymbol{K}}^\intercal \in \mathbb{R}^{m \times n}$ to keep the notation uncluttered. This definition of the subspace kernel sum rule follows from the kernel chain rule for tensor product features, where the conditional operator $C_{Y|X}$ is applied to the covariance embedding C_{XX}^π to obtain the covariance embedding C_{YX}^π (c.f. Equation). The subspace kernel sum rule follows from applying the transpose of the condition operator a second time from the right-hand side.

3.3 The Subspace Kernel Chain Rule

The subspace kernel chain rule is a straight forward modification of the kernel chain rule by Song et al (2013). We simply apply the subspace conditional operator $C_{Y|X}^S$ from the left side to a covariance operator $C_{XX}^\pi = \Upsilon_x \operatorname{diag}(\boldsymbol{\alpha}) \Upsilon_x^\intercal$ approximated from the weights $\boldsymbol{\alpha}$ of the prior mean map μ_X^π

$$C_{YX}^{S,\pi} = C_{Y|X}^S C_{XX}^\pi = \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \boldsymbol{L}_{\bar{\boldsymbol{K}}} \operatorname{diag}(\boldsymbol{\alpha}) \Upsilon_x^\intercal. \tag{29}$$

With the subspace kernel sum rule and the subspace kernel chain rule we can now construct the subspace kernel Bayes' rule.

3.4 The Subspace Kernel Bayes' Rule

The Bayes' rule computes a posterior distribution $P(X|Y)$ from a prior distribution $\pi(X)$ and a likelihood function $P(Y|X)$. Fukumizu et al (2013) derive a conditional operator $C_{X|Y}^\pi$ from the prior modified covariance operators C_{XY}^π and C_{YY}^π. We follow this approach and construct the subspace kernel Bayes' rule (subKBR) from the prior modified covariance operators $C_{XY}^{S,\pi}$ and $C_{YY}^{S,\pi}$ which we obtain from the subspace kernel chain rule and the subspace kernel sum rule for tensor product features, respectively. When applied to the embedding of a variate $\boldsymbol{y}$, the subspace kernel Bayes' rule returns the mean embedding of the conditional distribution $P(X|\boldsymbol{y})$ as

$$\mu_{X|y}^\pi = C_{X|Y}^{S,\pi} \boldsymbol{\phi}(\boldsymbol{y}) \tag{30}$$

$$\mu_{X|y}^\pi = C_{XY}^{S,\pi} \left(C_{YY}^{S,\pi}\right)^{-1} \boldsymbol{\phi}(\boldsymbol{y}). \tag{31}$$

From the subspace kernel chain rule, we obtain

$$C_{XY}^{S,\pi} = \left(C_{Y|X}^{S} C_{XX}^{\pi}\right)^{\intercal} \tag{32}$$

$$= \Upsilon_x \operatorname{diag}(\boldsymbol{\alpha}) \boldsymbol{L}_{\bar{\boldsymbol{K}}}^{\intercal} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal} \tag{33}$$

and from the subspace kernel sum rule

$$C_{YY}^{S,\pi} = C_{Y|X}^{S} C_{XX}^{\pi} \left(C_{Y|X}^{S}\right)^{\intercal} \tag{34}$$

$$= \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \boldsymbol{L}_{\bar{\boldsymbol{K}}} \operatorname{diag}(\alpha) \boldsymbol{L}_{\bar{\boldsymbol{K}}}^{\intercal} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal}. \tag{35}$$

To keep the notation of the subspace kernel Kalman rule uncluttered, we define the following matrices

$$\bar{\boldsymbol{\Lambda}} := \operatorname{diag}(\alpha) \boldsymbol{L}_{\bar{\boldsymbol{K}}}^{\intercal} \tag{36}$$

$$\bar{\boldsymbol{D}} := \boldsymbol{L}_{\bar{\boldsymbol{K}}} \operatorname{diag}(\alpha) \boldsymbol{L}_{\bar{\boldsymbol{K}}}^{\intercal} \in \mathbb{R}^{m \times m}, \tag{37}$$

$$\boldsymbol{E} := \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{G} \bar{\boldsymbol{K}} \in \mathbb{R}^{m \times m}, \tag{38}$$

where $\boldsymbol{G} = \boldsymbol{\Phi}_y^{\intercal} \boldsymbol{\Phi}_y$ is the kernel matrix of the samples $\boldsymbol{y}_i$. Using the same form of the Tikhonov regularization as the kernel Bayes' rule in Fukumizu et al (2011) and substituting the prior modified subspace covariance operators from Equations 33 and 35 results in

$$\mu_{X|y}^{\pi} = C_{XY}^{S,\pi} \left[\left(C_{YY}^{S,\pi}\right)^2 + \gamma \boldsymbol{I}\right]^{-1} C_{YY}^{S,\pi} \boldsymbol{\phi}(\boldsymbol{y}) \tag{39}$$

$$= \Upsilon_x \bar{\boldsymbol{\Lambda}} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal} \left[\left(\boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \bar{\boldsymbol{D}} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal}\right) \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \bar{\boldsymbol{D}} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal} + \gamma \boldsymbol{I}\right] \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \bar{\boldsymbol{D}} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{\Phi}_y^{\intercal} \boldsymbol{\phi}(\boldsymbol{y}) \tag{40}$$

$$= \Upsilon_x \bar{\boldsymbol{\Lambda}} \boldsymbol{E} \left[(\bar{\boldsymbol{D}} \boldsymbol{E})^2 + \gamma \boldsymbol{I}\right]^{-1} \bar{\boldsymbol{D}} \bar{\boldsymbol{K}}^{\intercal} \boldsymbol{g}_{\boldsymbol{y}}, \tag{41}$$

where we apply the matrix identity $\boldsymbol{A}\left(\boldsymbol{B}\boldsymbol{A} + \lambda \boldsymbol{I}\right)^{-1} = \left(\boldsymbol{A}\boldsymbol{B} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{A}$ with $\boldsymbol{A} = \boldsymbol{\Phi}_y \bar{\boldsymbol{K}}$ to obtain a finite matrix in the inverse. The term $\boldsymbol{g}_{\boldsymbol{y}} = \boldsymbol{\Phi}_y^{\intercal} \boldsymbol{\phi}(\boldsymbol{y})$ denotes the kernel vector of the new observation $\boldsymbol{y}$. Since $\boldsymbol{E}$ and $\bar{\boldsymbol{D}}$ are both in $\mathbb{R}^{m \times m}$, the matrix inversion is only in $O(m^3)$ instead of $O(n^3)$. The whole kernel Bayes' rule is in $O(nm^2)$ and, thus, scales linearly with the number of sample points (given a fixed reference set) instead of cubically as for the original kernel Bayes' rule.

3.5 Experimental Evaluation

We compare the performance, learning time and run time of the subspace kernel Bayes' filter in comparison to the standard kernel Bayes' filter with a simple toy task. We simulate a pendulum which we randomly initialize in the ranges $[0.1\pi, 0.4\pi]$ and $[-0.5\pi, 0.5\pi]$ for the angle θ and the angular velocity $\dot{\theta}$, respectively. The pendulum has a mass of 5kg and a friction coefficient of 1. We apply Gaussian white noise to the system with a variance of 1, and to the observations with a variance of 0.1. Additionally, the observed angles are randomly perturbed by an offset of $\pi/4$. These random perturbations occur with a probability of 0.1 in every time step. Each episode consists of 30 time steps with $\Delta t = 0.1$.

Figure 1 shows that the subspace KBF has a slightly better performance when the training set equals the subspace set and maintains the performance of the standard KBF with an increasing number of training samples while the subspace set is fixed to 100 samples.

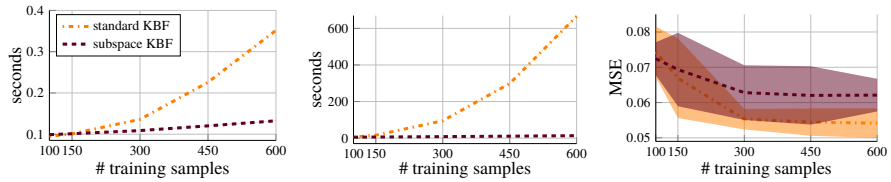


Fig. 1: The subspace variant of the kernel Bayes' filter outperforms the standard kernel Bayes' filter in both, the tarining time depicted in the left plot and the run time depicted in the middle plot, while maintaining a similar performance to the standard kernel Bayes' rule as depicted in the right plot. The size of the subspace is fixed to 100 samples.

However, at the same time the learning time of the subspace KBF increases at a much lower magnitude and the run time is nearly constant while the learning and run time of the standard KBR grow cubically. The samples for the subspace kernel Bayes rule are drawn uniformly without replacement from the full sample set.

4 The Kernel Kalman Rule

In this section we will present the *kernel Kalman rule* (KKR) as an alternative to the kernel Bayes' rule (Fukumizu et al 2011). We assume a prior belief state over a variable X embedded in a Hilbert space $\mathcal{H}_k$ as

$$\boldsymbol{\mu}_{X,t}^{-} = \mathbb{E}_{X_t|\boldsymbol{y}_{1:t-1}}[\boldsymbol{\varphi}(X)]$$

and new measurement $\boldsymbol{y}_t$ embedded in a Hilbert space $\mathcal{H}_g$ as $\boldsymbol{\phi}(\boldsymbol{y}_t)$. With the kernel Kalman rule we want to infer the embedding of the posterior belief state

$$\boldsymbol{\mu}_{X,t}^{+} = \mathbb{E}_{X_t|\boldsymbol{y}_{1:t}}[\boldsymbol{\varphi}(X)] \in \mathcal{H}_k$$

from the prior belief and the new measurement. The derivations for the KKR are inspired by the ansatz from recursive least squares (Gauss 1823; Sorenson 1970; Simon 2006), and thus the resulting update equations follow from a clear optimality criterion.

4.1 Derivation using Recursive Least Squares

Let $C_{Y|X}$ be a conditional embedding operator of the observation model $P(Y|X)$ that yields for a given belief state embedded in the Hilbert space $\mathcal{H}_k$ the distribution over possible observations embedded into the Hilbert space $\mathcal{H}_g$. We call this conditional embedding operator also *observation operator*. The objective of the KKR is then to find the mean embedding $\boldsymbol{\mu}_X$ that minimizes the squared error

$$\sum_t \left(\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X}\boldsymbol{\mu}_X\right)^T \mathcal{R}^{-1} \left(\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X}\boldsymbol{\mu}_X\right), \tag{42}$$

with some metric $\mathcal{R}$, in an iterative fashion. In each iteration, we want to update the prior mean map $\boldsymbol{\mu}_{X,t}^{-}$ based on the measurement $\boldsymbol{y}_t$ to obtain the posterior mean map $\boldsymbol{\mu}_{X,t}^{+}$.

From the recursive least squares solution, we know that the update rule for obtaining the posterior mean map $\boldsymbol{\mu}^+_{X,t}$ is

$$\boldsymbol{\mu}^+_{X,t} = \boldsymbol{\mu}^-_{X,t} + Q_t \left(\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X} \boldsymbol{\mu}^-_{X,t} \right), \tag{43}$$

where Q_t is the Hilbert space *Kalman gain operator* that is applied to the correction term $\boldsymbol{\delta}_t = \boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X} \boldsymbol{\mu}^-_{X,t}$. We call this rule the *kernel Kalman rule* (KKR). It remains to show how to obtain the Kalman gain operator. As we will show in the next paragraphs that this rule is an unbiased estimator of the posterior mean map, we cannot obtain the optimal Q_t by minimizing the error directly. Instead, we will show in Section 4.1.2, how we can find an optimal Q_t by minimizing the covariance of the error instead.

4.1.1 The Kernel Kalman Update is an Unbiased Estimator of the Posterior Mean Map

The embedding of the observation $\boldsymbol{\phi}(\boldsymbol{y}_t)$ in the correction term $\boldsymbol{\delta}_t$ is a single-sample estimator of the embedding of the true distribution over observations $\boldsymbol{\mu}^-_{Y,t}$. We make the assumption that the error of this single-sample estimator is independent from the state x_t, as we can obtain the embedding of the distribution over the observations from the observation operator and the prior distribution $\boldsymbol{\mu}^-_{X,t}$ as

$$\boldsymbol{\phi}(\boldsymbol{y}_t) = \boldsymbol{\mu}^-_{Y|X,t} + \boldsymbol{\zeta}_t = C_{Y|X} \boldsymbol{\mu}^-_{X,t} + \boldsymbol{\zeta}_t, \tag{44}$$

where $\boldsymbol{\zeta}_t$ denotes the error of the single sample estimator to the embedding of the true distribution. By taking the expectation it is easy to show that the error of the single-sample estimator is zero-mean and thus $\boldsymbol{\phi}(\boldsymbol{y}_t)$ is an unbiased estimator for $\boldsymbol{\mu}^-_{Y|X,t}$. We refer to Appendix B.1 for a more detailed derivation. Similarly, the error of the a-posteriori estimator to the embedding of the true state is given as

$$\begin{aligned} \boldsymbol{\varepsilon}^+_t &= \boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}^+_{X,t} && (45) \\ &= \boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}^-_{X,t} - Q_t(\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X} \boldsymbol{\mu}^-_{X,t}), && (46) \end{aligned}$$

where we use Equation 43 to substitute the embedding of the posterior belief. By substituting $\boldsymbol{\phi}(\boldsymbol{y}_t)$ with Equation 44 and defining the error of the a-priori estimator analogously as $\boldsymbol{\varepsilon}^-_t = \boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}^-_{X,t}$, we arrive at

$$\begin{aligned} \boldsymbol{\varepsilon}^+_t &= \boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}^-_{X,t} - Q_t(C_{Y|X} \boldsymbol{\varphi}(\boldsymbol{x}_t) + \boldsymbol{\zeta}_t - C_{Y|X} \boldsymbol{\mu}^-_{X,t}) && (47) \\ &= \left(\mathcal{I} - Q_t C_{Y|X} \right) (\boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}^-_{X,t}) - Q_t \boldsymbol{\zeta}_t && (48) \\ &= \left(\mathcal{I} - Q_t C_{Y|X} \right) \boldsymbol{\varepsilon}^-_t - Q_t \boldsymbol{\zeta}_t && (49) \end{aligned}$$

with identity operator $\mathcal{I}$. We can now apply the expectation operator and exploit its linearity to obtain

$$\begin{aligned} \mathbb{E}\left[\boldsymbol{\varepsilon}^+_t \right] &= \mathbb{E}\left[\left(\mathcal{I} - Q_t C_{Y|X} \right) \boldsymbol{\varepsilon}^-_t - Q_t \boldsymbol{\zeta}_t \right] \\ &= \left(\mathcal{I} - Q_t C_{Y|X} \right) \mathbb{E}\left[\boldsymbol{\varepsilon}^-_t \right] - Q_t \mathbb{E}\left[\boldsymbol{\zeta}_t \right]. && (50) \end{aligned}$$

Since the residual of the observation operator is zero mean ($\mathbb{E}[\boldsymbol{\zeta}_t] = 0$), we see that, given an unbiased a-priori estimator ($\mathbb{E}[\boldsymbol{\varepsilon}^-_t] = 0$), the a-posteriori estimator obtained from the kernel Kalman update is unbiased ($\mathbb{E}[\boldsymbol{\varepsilon}^+_t] = 0$) independent of the choice of Q. Thus, we cannot use the expected error of the estimator as an optimality criterion for the Kalman gain operator.

4.1.2 Finding the Optimal Kernel Kalman Gain Operator

If the expected error—or the first moment of the error distribution—is already zero, taking the covariance of the error—or the second moment—is a consequent choice. Hence, we chose the kernel Kalman gain operator $\mathcal{Q}_t$ which minimizes the expected squared loss $\mathbb{E}\left[\left(\boldsymbol{\varepsilon}_t^+\right)^\intercal \boldsymbol{\varepsilon}_t^+\right]$ or equivalently the variance of the estimator. The objective for minimizing the variance can also be reformulated as minimizing the trace of the a-posteriori covariance operator $\mathcal{C}_{XX,t}^+$ of the state $\boldsymbol{x}_t$ at time t, i.e., $\min_{\mathcal{Q}_t} \mathbb{E}\left[\left(\boldsymbol{\varepsilon}_t^+\right)^\intercal \boldsymbol{\varepsilon}_t^+\right] = \min_{\mathcal{Q}_t} \operatorname{Tr} \mathcal{C}_{XX,t}^+$. Using the formulation of the posterior error from Equation 49 and the independence assumption of $\boldsymbol{\zeta}_t$ and $\boldsymbol{\varepsilon}_t$ allows us to reformulate the a-posteriori covariance operator as

$$\mathcal{C}_{XX,t}^+ = \left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right) \mathcal{C}_{XX,t}^- \left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)^\intercal + \mathcal{Q}_t \mathcal{R} \mathcal{Q}_t^\intercal, \tag{51}$$

where $\mathcal{R} = \mathbb{E}[\zeta_t \zeta_t^\intercal]$ is the covariance of the residual of the observation operator. Taking the derivative of the trace of the covariance operator and setting it to zero leads to the solution for the kernel Kalman gain operator

$$\mathcal{Q}_t = \mathcal{C}_{XX,t}^- \mathcal{C}_{Y|X}^\intercal \left(\mathcal{C}_{Y|X} \mathcal{C}_{XX,t}^- \mathcal{C}_{Y|X}^\intercal + \mathcal{R}\right)^{-1}. \tag{52}$$

We provide a detailed derivation of the optimal Kalman gain operator in Appendix B.2. From Equations 51 and 52, we can also see that it is possible to recursively estimate the covariance embedding operator independently of the mean map or of the observations. This property will allow us later to precompute the covariance embedding operator as well as the kernel Kalman gain operator to further improve the computational complexity of our algorithm. Following Simon (2006), the update of the covariance operator can be further simplified to

$$\mathcal{C}_{XX,t}^+ = \mathcal{C}_{XX,t}^- - \mathcal{Q}_t \mathcal{C}_{Y|X} \mathcal{C}_{XX,t}^-. \tag{53}$$

The derivations of this simplification can be found in Appendix B.3. In the following section we will show how to obtain the empirical Kalman update rule from a finite data set.

4.2 Empirical Kernel Kalman Rule

The equations for the kernel Kalman rule that we derived in the previous section are based on embeddings in infinite dimensional Hilbert spaces and operators that map between these spaces. In practice, these embeddings and operators are estimated from a finite set of samples $\mathcal{D} = \left[(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n)\right]$. In this section we will show how we can reformulate the kernel Kalman rule with manipulations of finite matrices by applying the kernel trick, i.e., matrix identities. Based on the data set $\mathcal{D}$ and the corresponding feature matrices $\boldsymbol{\Upsilon}_x$ and $\boldsymbol{\Phi}_y$, the finite sample estimators of the prior mean embedding and the prior covariance operator are given as

$$\hat{\boldsymbol{\mu}}_{X,t}^- = \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^- \qquad \text{and} \qquad \hat{\mathcal{C}}_{XX,t}^- = \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{\Upsilon}_x^\intercal, \tag{54}$$

respectively, with weight vector $\boldsymbol{m}_t^-$ and positive definite weight matrix $\boldsymbol{S}_t^-$. Using this finite sample estimator of the covariance operator, the finite sample estimator of the conditional operator from Equation 6, and by approximating the covariance of the residual of the observation operator with a small diagonal $\mathcal{R} = \kappa \mathcal{I}$, we can rewrite the kernel Kalman gain operator as

$$\mathcal{Q}_t = \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{O}^\intercal \boldsymbol{\Phi}_y^\intercal \left(\boldsymbol{\Phi}_y \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal \boldsymbol{\Phi}_y^\intercal + \kappa \mathcal{I}\right)^{-1}, \tag{55}$$

with the observation matrix $\boldsymbol{O} = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1}\boldsymbol{K}$. Here, the approximation of $\mathcal{R} = \kappa \mathcal{I}$ also acts as a small regularization in the inverse to ensure its positive definiteness and to improve the numerical stability of the kernel Kalman rule. However, $\mathcal{Q}_t$ still requires the inversion of an infinite dimensional matrix. Using matrix identities, we can solve this problem and arrive at

$$\mathcal{Q}_t = \Upsilon_x \underbrace{\boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\boldsymbol{G}\boldsymbol{O}\boldsymbol{S}_t^- \boldsymbol{O}^\intercal + \kappa \boldsymbol{I})^{-1}}_{\boldsymbol{Q}_t} \Phi_y^\intercal, \tag{56}$$

where we defined $\boldsymbol{Q}_t = \boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\boldsymbol{G}\boldsymbol{O}\boldsymbol{S}_t^- \boldsymbol{O}^\intercal + \kappa \boldsymbol{I})^{-1} \in \mathbb{R}^{n \times n}$ with the Gram matrix of the observations $\boldsymbol{G} = \boldsymbol{\Phi}_y^\intercal \boldsymbol{\Phi}_y$. Based on this reformulation of the kernel Kalman gain operator, we can obtain finite vector/matrix representations of the update equations for the estimator of the mean embedding (Equation 43) and the estimator of the covariance operator (Equation 53). For the weight vector $\boldsymbol{m}_t$, we arrive at

$$\boldsymbol{m}_t^+ = \boldsymbol{m}_t^- + \boldsymbol{Q}_t \left(\boldsymbol{g}_{:\boldsymbol{y}_t} - \boldsymbol{G}\boldsymbol{O}\boldsymbol{m}_t^- \right), \tag{57}$$

where $\boldsymbol{g}_{:\boldsymbol{y}_t} = \boldsymbol{\Phi}_y^\intercal \boldsymbol{\phi}(\boldsymbol{y}_t)$ is the embedding of the measurement at time t. Similarly, we can also obtain the update equation for the weight matrix $\boldsymbol{S}_t$ as

$$\boldsymbol{S}_t = \boldsymbol{S}_t^- - \boldsymbol{Q}_t \boldsymbol{G}\boldsymbol{O}\boldsymbol{S}_t^-. \tag{58}$$

The algorithm requires the inversion of a $m \times m$ matrix in every iteration for computing the kernel Kalman gain matrix $\boldsymbol{Q}_t$. Hence, similar to the kernel Bayes' rule, the computational complexity of a straightforward implementation would scale cubically with the number of data points m. However, in contrast to the KBR, the inverse in $\boldsymbol{Q}_t$ is only dependent on time and not on the estimate of the mean map. Thus, the kernel Kalman gain matrix can be precomputed since it is identical for multiple parallel runs of the algorithm. Furthermore, if the stream of incoming measurements is reliable (no time steps without incoming measurement), $\boldsymbol{S}_t$ will converge to a stationary matrix and by that $\boldsymbol{Q}_t$ will become stationary as well. While many applications do not require to perform state estimations in parallel, it is a huge advantage for hyper-parameter optimization as we can evaluate multiple trajectories from a validation set simultaneously. As for most kernel-based methods, hyper-parameter optimization is crucial for scaling the approach to complex tasks. So far, the hyper-parameters of the kernels for the KBF have typically been set by heuristics as optimization would be too expensive.

4.3 The Subspace Kernel Kalman Rule

In Section 3 we have already shown how we can apply the subspace conditional embedding operator to leverage from large data sets but at the same time maintain the computational tractability of the learned models. In this section, we will now show how we can apply this technique to the kernel Kalman rule to obtain the *subspace kernel Kalman rule* (subKKR).

A core difference between the KKR and the subKKR is the representation of the embedded distributions. While we represent the embeddings for the kernel Kalman rule as weight vector $\boldsymbol{m}_t^-$ and weight matrix $\boldsymbol{S}_t^-$, we use the projections into a subspace

$$\boldsymbol{n}_t = \boldsymbol{\Gamma}_x^\intercal \mu_t = \boldsymbol{\Gamma}_x^\intercal \Upsilon_x \boldsymbol{m}_t = \bar{\boldsymbol{K}}^\intercal \boldsymbol{m}_t, \tag{59}$$

$$\boldsymbol{P}_t = \boldsymbol{\Gamma}_x^\intercal C_{XX,t} \boldsymbol{\Gamma}_x = \boldsymbol{\Gamma}_x^\intercal \Upsilon_x \boldsymbol{S}_t \Upsilon_x^\intercal \boldsymbol{\Gamma}_x = \bar{\boldsymbol{K}}^\intercal \boldsymbol{S}_t \bar{\boldsymbol{K}}. \tag{60}$$

to represent the distribution for the subspace kernel Kalman rule. These projections will later allow us to express all operations in the lower dimensional subspace instead of the

space spanned by the full data set. Matrix manipulations with the full data set are then only necessary during the learning phase of the KKR not while performing inference.

We use a slightly modified version of the kernel Kalman gain from Equation 52 where we approximate the covariance operator $\mathcal{R}$ with a diagonal matrix $\kappa\mathcal{I}$. With the subspace conditional embedding operator of the distribution $P(Y|X)$, as derived in Section 2.1.2

$$C^S_{Y|X} = \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{\Gamma}_x^\intercal, \tag{61}$$

we obtain the subspace kernel Kalman gain operator as

$$\mathcal{Q}^S_t = C^-_{XX,t} \left(C^S_{Y|X} \right)^\intercal \left(C^S_{Y|X} C^-_{XX,t} \left(C^S_{Y|X} \right)^\intercal + \kappa \mathcal{I} \right)^{-1} \tag{62}$$

We can further derive a finite matrix representation of the operator using matrix identities and the projection into the subspace spanned by the features $\boldsymbol{\Gamma}_x^\intercal$ as

$$\boldsymbol{\Gamma}_x^\intercal \mathcal{Q}^S_t = \underbrace{\boldsymbol{P}_t^- (\boldsymbol{O}^S)^\intercal \left(\bar{\boldsymbol{K}}^\intercal \boldsymbol{G} \bar{\boldsymbol{K}} \boldsymbol{O}^S \boldsymbol{P}_t^- (\boldsymbol{O}^S)^\intercal + \kappa \boldsymbol{I} \right)^{-1} \bar{\boldsymbol{K}}^\intercal}_{\boldsymbol{Q}^S_t} \boldsymbol{\Phi}_y^\intercal, \tag{63}$$

where we define the subspace kernel Kalman gain matrix $\boldsymbol{Q}^S_t$ using the short hand $\boldsymbol{O}^S := (\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I})^{-1}$. A detailed derivation can be found in Appendix B.5. Note that $\boldsymbol{Q}^S_t \in \mathbb{R}^{m \times n}$ and not $\mathbb{R}^{m \times m}$, however when applying the subspace KKR in an inference algorithm, we can use the matrix $\bar{\boldsymbol{K}}^\intercal$ on the right side as a projection for the high-dimensional embedding of the distribution over the variable Y to which the gain is applied (see Algorithm 2 for an example). From here, the update equation for the projection of the mean map becomes

$$\boldsymbol{n}^+_{X,t} = \boldsymbol{n}^-_{X,t} + \boldsymbol{Q}^S_t \left(\boldsymbol{g}_{:\boldsymbol{y}_t} - \boldsymbol{G} \bar{\boldsymbol{K}} \boldsymbol{O}^S \boldsymbol{n}^-_t \right), \tag{64}$$

And similarly, we can derive the update equation for the covariance embedding as

$$\boldsymbol{P}^+_t = \boldsymbol{P}^-_t - \boldsymbol{Q}^S_t \boldsymbol{G} \bar{\boldsymbol{K}} \boldsymbol{O}^S \boldsymbol{P}^-_t \tag{65}$$

In contrast to the kernel Kalman gain presented in the previous section, but also in contrast to the kernel Bayes rules discussed in Section 2.1.5, the subspace kernel Kalman gain requires only the inversion of an $m \times m$ matrix instead of an $n \times n$ matrix, where $m \ll n$. Still, the full data set of n samples can be used to learn the Kalman gain operator.

4.4 Experimental Comparison of (sub)KKR and (sub)KBR

We compare the performance of the (subspace) kernel Kalman rule to the performance of the (subspace) kernel Bayes rule on a simple stationary filtering task for estimating the expectation of a Gaussian distribution. We sample 500 context variables c_i uniformly from the interval $[-5, 5]$ as the mean of the Gaussian distributions. Afterwards we draw one single sample s_i for each context from $\mathcal{N}(c_i, \frac{1}{3})$ and learn the kernel Kalman rule and the different versions of the kernel Bayes rule with the context variables as states and the samples as observations. For the performance comparison (Figure 2), the KKR and the KBR are learned with a kernel size of 200 samples, subKKR and subKBR are both learned with 200 samples to span the subspace and the full set of 500 samples to learn the operators. In the comparison of time efficiency, the respective kernel size and subspace size is denoted as column header,

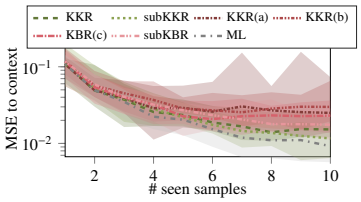


Fig. 2: Performance of the KKR updates vs the KBR updates for estimating the mean of a Gaussian distribution with 1-10 seen samples. The ML estimate serves as a baseline. Depicted are the median and the $(0.15, 0.85)$-quantiles of the MSE to the true mean over 20 runs.

#	200	300	400	500
KKR	**0.1110**	**0.3360**	**0.7155**	**1.3965**
subKKR	**0.1820**	**0.4655**	**0.9130**	**1.6075**
KBR(a)	2.9395	9.2395	21.5035	41.6955
KBR(b)	0.8695	2.5840	5.7580	10.9900
KBR(c)	0.5455	1.5575	3.3695	6.4465
subKBR	2.3530	4.7625	7.6540	12.7960

Table 1: Time consumptions of the KKR and KBR update methods for different kernel sizes. The update was performed on 10 samples from 10 different distributions. The subspace KKR and KBR updates are trained with 500 samples in the full data set. Both KKR methods outperform the KBR methods clearly as they are able to process the update on the 10 different distributions in parallel.

while the subKKR and subKBR have always been learned with the full data set of 500 samples. All data were drawn uniformly without replacement from the full data set.

To evaluate the methods, we generate a data set with ten context variables from the same uniform distribution. Now, we draw ten samples from the Gaussian around each context and update each method iteratively with these ten samples. For each update we compute the squared error to the true context and take the mean over all ten context variables. We use squared exponentials as kernel functions and optimize their bandwidths as well as the regularization parameters using CMA-ES (Hansen 2006). Figure 2 shows the median and the $(0.15, 0.85)$-quantiles of the MSE to the true context over the number of seen samples. As a baseline we depict the maximum-likelihood (ML) estimate of the expectation. We see that while in the beginning all methods perform similar to the ML estimate, with more seen samples KKR and subKKR outperform all variants of the KBR. Moreover the choice to depict the median and $(0.15, 0.85)$-quantiles over mean and standard deviation is due to the instable optimization behavior of the KBR which produced a lot of outliers. In Table 1 we state the time consumed to perform ten KKR/KBR updates on 10 estimation tasks for different kernel sizes. Here, the KKR/subKKR methods benefit from their ability to process the updates for all 10 estimation tasks in parallel. Yet, the ability to precompute $\boldsymbol{Q}_t/\boldsymbol{Q}_t^S$ and $\boldsymbol{S}_t/\boldsymbol{P}_t$ is not even exploited.

5 Applications of the Kernel Kalman Rule

In Section 4, we have shown how we can derive the KKR as an alternative operator for Bayesian updates in the framework for nonparametric inference. In this section we will present two applications of the kernel Kalman rule. In Section 5.1 we will first present the *kernel Kalman filter* (KKF) and discuss details about the implementation. A subspace variate of the KKF is presented in Section 5.2 and experimental results of both are shown in Section 5.3. The *kernel forward backward smoother* (KFBS) is presented as another application of the KKR in Section 5.4 and a subspace variate is discussed in Section 5.5. We finally show experimental evaluations of the KFBS and the subKFBS in Section 5.6.

5.1 The Kernel Kalman Filter

Similar to the kernel Bayes' filter (Fukumizu et al 2013; Song et al 2013), we can combine the kernel Kalman rule with the kernel sum rule to formulate the *kernel Kalman filter* (KKF). To learn the models of the KKF, we assume a data set $\mathcal{D} = \{(\tilde{x}_1, x_1, y_1), \ldots, (\tilde{x}_m, x_m, y_m)\}$ consisting of triples with preceding state $\tilde{x}_i$, state x_i, and measurement y_i as given. We further assume the states to be Markov, i.e., the state x_i is only dependent on its predecessor $\tilde{x}_i$. Based on this data set we define the feature matrices $\Upsilon_x = [\boldsymbol{\varphi}(x_1), \ldots, \boldsymbol{\varphi}(x_m)]$, $\Upsilon_{\tilde{x}} = [\boldsymbol{\varphi}(\tilde{x}_1), \ldots, \boldsymbol{\varphi}(\tilde{x}_m)]$, and $\boldsymbol{\Phi}_y = [\boldsymbol{\phi}(y_1), \ldots, \boldsymbol{\phi}(y_m)]$. In contrast to the KBF, we represent the belief state as mean map $\mu_{X,t} = \Upsilon_x \boldsymbol{m}_t$ and as covariance operator $C_{XX,t} = \Upsilon_x \boldsymbol{S}_t \Upsilon_x^\intercal$.

The forward model $P(X|\tilde{X})$ that propagates the posterior belief state at time t to the prior belief state at time $t+1$ can then be learned as conditional embedding operator

$$C_{X|\tilde{X}} = \Upsilon_x \left(\boldsymbol{K}_{\tilde{x}\tilde{x}} + \lambda \boldsymbol{I}\right)^{-1} \Upsilon_{\tilde{x}}^\intercal, \tag{66}$$

which we also call transition operator. Here, $\boldsymbol{K}_{\tilde{x}\tilde{x}}$ is the Gram matrix of the features of the preceding states $\Upsilon_{\tilde{x}}$. The posterior belief state at time t is then propagated to the prior belief state at time $t+1$ time by applying the kernel sum rule. That is, we apply the transition operator to the posterior mean map and the posterior covariance embedding at time t and obtain prior mean map and prior covariance embedding at time $t+1$, i.e.,

$$\mu_{X,t+1} = C_{X|\tilde{X}} \mu^+_{X,t} = \Upsilon_x \boldsymbol{T} \boldsymbol{m}^+_t, \qquad \Leftrightarrow \qquad \boldsymbol{m}^-_{t+1} = \boldsymbol{T} \boldsymbol{m}^+_t \tag{67}$$

$$C^-_{XX,t+1} = C_{X|\tilde{X}} C^+_{XX,t} C^\intercal_{X|\tilde{X}} + \mathcal{V} \qquad \Leftrightarrow \qquad \boldsymbol{S}^-_{t+1} = \boldsymbol{T} \boldsymbol{S}^+_t \boldsymbol{T}^\intercal + \boldsymbol{V}. \tag{68}$$

Note that the propagation of the covariance embedding is slightly different to the kernel sum rule by Song et al (2013), however this formulation follows directly from the kernel chain rule (c.f. Equations 28 and 11). Analog to the observation matrix $\boldsymbol{O}$ (c.f. Section 4.2), we denote the transition matrix $\boldsymbol{T} = (\boldsymbol{K}_{\tilde{x}\tilde{x}} + \lambda_{\boldsymbol{T}} \boldsymbol{I})^{-1} \boldsymbol{K}_{\tilde{x}x}$, where $\boldsymbol{K}_{\tilde{x}x} = \Upsilon_{\tilde{x}}^\intercal \Upsilon_x$ is the kernel matrix of the preceding states and the states. The covariance of the transition residual $\mathcal{V}$ and its finite matrix representation $\boldsymbol{V}$ can be obtained as

$$\mathcal{V} = \frac{1}{m} \left(C_{X|\tilde{X}} \Upsilon_x - \Upsilon_{\tilde{x}}\right) \left(C_{X|\tilde{X}} \Upsilon_x - \Upsilon_{\tilde{x}}\right)^\intercal \tag{69}$$

$$= \frac{1}{m} \left(\Upsilon_x \left(\boldsymbol{K}_{\tilde{x}\tilde{x}} + \lambda \boldsymbol{I}\right)^{-1} \Upsilon_{\tilde{x}}^\intercal \Upsilon_x - \Upsilon_{\tilde{x}}\right) \left(\Upsilon_x \left(\boldsymbol{K}_{\tilde{x}\tilde{x}} + \lambda \boldsymbol{I}\right)^{-1} \Upsilon_{\tilde{x}}^\intercal \Upsilon_x - \Upsilon_{\tilde{x}}\right)^\intercal \tag{70}$$

$$= \Upsilon_{\tilde{x}} \underbrace{\frac{1}{m} \left(\left(\boldsymbol{K} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{K} - \boldsymbol{I}\right) \left(\left(\boldsymbol{K} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{K} - \boldsymbol{I}\right)^\intercal}_{\boldsymbol{V}} \Upsilon_{\tilde{x}}^\intercal \tag{71}$$

On the new prior belief state that we obtain from the transition update, we can afterwards apply the kernel Kalman rule as observation update. Before we give a condensed summary of the kernel Kalman filter in Algorithm 1, we will discuss how we obtain the embedding of the distribution over the initial states in the next section. To extract some meaningful information from the RKHS-embedded distributions, we furthermore need to find mapping of the embedded distribution back into the state space. In Section 5.1.2, show how we approached the so-called *preimage problem* and shortly discuss other solutions.

5.1.1 Embedding the Initial State Distribution

Before running the filter on incoming measurements y_t, we need to initialize the belief state with an initial mean map $\mu_{X,0}$ and an initial covariance operator $C_{XX,0}$. We can obtain these inital embeddings from a data set $\mathcal{D}_0 = \{x_1^0, \dots, x_N^0\}$ which consists in general of samples from the initial distribution of the system. Practically, we can obtain this data set by taking the inital states from multiple training episodes or—if we assume a stationary distribution—we can also take all training samples for the initialization. We can obtain the initial mean map by first embedding a uniform distribution into the RKHS spanned by the features of the initial states $\boldsymbol{\Upsilon}_{x,0}$. Afterwards, we apply a conditional operator to map this distribution into the Hilbert space spanned by the features $\boldsymbol{\Upsilon}_x$ as

$$\mu_{X,0} = \boldsymbol{\Upsilon}_x \boldsymbol{m}_0 = \boldsymbol{\Upsilon}_x (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_{x,0} \mathbf{1}_N \frac{1}{N} \tag{72}$$

$$\Leftrightarrow \quad \boldsymbol{m}_0 = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}_0 \mathbf{1}_N \frac{1}{N}. \tag{73}$$

where $\boldsymbol{K}_0 = \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_{x,0}$ is the kernel matrix of the training samples and the samples in $\mathcal{D}_0$, and $\mathbf{1}_N$ denotes the N-dimensional all-ones vector. Similarly, we can obtain the initial covariance embedding operator as

$$C_{XX,0} = \boldsymbol{\Upsilon}_x \boldsymbol{S}_0 \boldsymbol{\Upsilon}_x^\intercal = \frac{1}{N} \boldsymbol{\Upsilon}_x (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}_0 \boldsymbol{K}_0^\intercal (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{\Upsilon}_x^\intercal - \boldsymbol{\Upsilon}_x \boldsymbol{m}_0 \boldsymbol{m}_0^\intercal \boldsymbol{\Upsilon}_x^\intercal. \tag{74}$$

Hence, we can obtain the initial weight vector $\boldsymbol{m}_0$ and the initial weight matrix $\boldsymbol{S}_0$ by computing the mean and the covariance over the columns of the matrix $\boldsymbol{C}_0 = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}_0$.

5.1.2 The Pre-image Problem / Recovering the State-Space Distribution

Recovering a distribution in the state space that is a pre-image of a given mean map is still a topic of ongoing research. There are several approaches to this problem, such as fitting a Gaussian mixture model (Mccalman et al 2013), or sampling from the embedded distribution by optimization (Chen et al 2012). In the experiments conducted for this paper, we approach the preimage problem by matching a Gaussian distribution, which is a reasonable choice if the recovered distribution is unimodal. Since we embed the belief state for the kernel Kalman rule as a mean map and as a covariance operator, obtaining the mean and covariance of a Gaussian approximation is done by simple matrix manipulations. The space of the samples $\mathbb{R}^d$ together with the linear kernel $k(\boldsymbol{x}_1, \boldsymbol{x}_2) = \langle \boldsymbol{x}_1, \boldsymbol{x}_2 \rangle = \boldsymbol{x}_1^\intercal \boldsymbol{x}_2$ forms an RKHS as well. Therfore, we can simply define a conditional embedding operator that maps from the Hilbert space of the feature vectors to the Hilbert space of the samples as

$$C_{\text{pre}} = \boldsymbol{X} (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{\Upsilon}_x^\intercal. \tag{75}$$

By applying this conditional operator now to the belief state, we obtain the mean of the embedded distribution in the sample space

$$\boldsymbol{\eta}_t = C_{\text{pre}} \boldsymbol{\mu}_{X,t} = C_{\text{pre}} \mathbb{E}_{b_t}[\boldsymbol{\varphi}(X)] = \mathbb{E}_{b_t}[C_{\text{pre}} \boldsymbol{\varphi}(X)] = \mathbb{E}_{b_t}[X]. \tag{76}$$

Similarly, we can also apply this operator to the covariance embedding to obtain the covariance of the belief state in the sample space

$$\begin{aligned}\boldsymbol{\Sigma}_t &= C_{\text{pre}} C_{XX,t} C_{\text{pre}}^\intercal \\ &= C_{\text{pre}} \left(\mathbb{E}_{b_t} \left[\boldsymbol{\varphi}(X) \otimes \boldsymbol{\varphi}(X) \right] - \boldsymbol{\mu}_{X,t} \otimes \boldsymbol{\mu}_{X,t} \right) C_{\text{pre}}^\intercal \\ &= \mathbb{E}_{b_t} \left[C_{\text{pre}} \boldsymbol{\varphi}(X) \otimes \boldsymbol{\varphi}(X) C_{\text{pre}}^\intercal \right] - C_{\text{pre}} \boldsymbol{\mu}_{X,t} \otimes \boldsymbol{\mu}_{X,t} C_{\text{pre}}^\intercal \\ &= \mathbb{E}_{b_t} \left[X \otimes X \right] - \boldsymbol{\eta}_t \otimes \boldsymbol{\eta}_t^\intercal \end{aligned} \tag{77}$$

However, also any other approach from the literature can be used in the kernel Kalman filter algorithm.

Algorithm 1: The Kernel Kalman Filter

input: triples $\{(\tilde{x}_1, x_1, y_1), \ldots, (\tilde{x}_m, x_m, y_m)\}$,
kernel functions k and g, regularization parameters λ and κ,
let $\boldsymbol{X}$ be the matrix of all x_i as columns, $\tilde{\boldsymbol{X}}$ and $\boldsymbol{Y}$ analogously,
let $\boldsymbol{X}_0$ be the matrix of all data points used to compute the initial embeddings

compute kernel matrices
$\boldsymbol{K} = k(\boldsymbol{X}, \boldsymbol{X})$, $\boldsymbol{K}_{\tilde{x}\tilde{x}} = k(\tilde{\boldsymbol{X}}, \tilde{\boldsymbol{X}})$, $\boldsymbol{K}_{\tilde{x}x} = k(\tilde{\boldsymbol{X}}, \boldsymbol{X})$, and $\boldsymbol{G} = g(\boldsymbol{Y}, \boldsymbol{Y})$

compute model matrices
$\boldsymbol{T} = (\boldsymbol{K}_{\tilde{x}\tilde{x}} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}_{\tilde{x}x}$ and $\boldsymbol{O} = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}$

compute initial embeddings
kernel matrix with samples of the initial distribution: $\boldsymbol{K}_0 = k(\boldsymbol{X}, \boldsymbol{X}_0)$, $\boldsymbol{C}_0 = (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \boldsymbol{K}_0$,
compute mean and variance over the columns: $\boldsymbol{m}_0 = \text{mean}(\boldsymbol{C}_0)$, $\boldsymbol{S}_0 = \text{var}(\boldsymbol{C}_0)$

loop

  if *new observation* $\boldsymbol{y}_t$ *available* **then**

    compute kernel Kalman gain
    $\boldsymbol{Q}_t = \boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\boldsymbol{G}\boldsymbol{O}\boldsymbol{S}_t^- \boldsymbol{O}^\intercal + \kappa \boldsymbol{I})^{-1}$

    innovation update
    $\boldsymbol{m}_t^+ = \boldsymbol{m}_t^- + \boldsymbol{Q}_t (\boldsymbol{g}_{:y_t} - \boldsymbol{G}\boldsymbol{O}\boldsymbol{m}_t^-)$
    $\boldsymbol{S}_t^+ = \boldsymbol{S}_t^- - \boldsymbol{Q}_t \boldsymbol{G}\boldsymbol{O}\boldsymbol{S}_t^-$

  transition update
  $\boldsymbol{m}_{t+1}^- = \boldsymbol{T}\boldsymbol{m}_t^+, \quad \boldsymbol{S}_{t+1}^- = \boldsymbol{T}\boldsymbol{S}_t^+ \boldsymbol{T}^\intercal + \boldsymbol{V}$

  project into state space
  $\boldsymbol{\eta}_t = \boldsymbol{X}\boldsymbol{O}\boldsymbol{m}_t, \quad \boldsymbol{\Sigma}_t = \boldsymbol{X}\boldsymbol{O}\boldsymbol{S}_t\boldsymbol{O}^\intercal\boldsymbol{X}^\intercal$

5.1.3 Embedding Observation Windows

So far, we assumed that we have access to the latent states x_i in our training set. However, in many setups we only have access to the partial observations y_i which do not have the Markov property. Yet, we can still learn a KKR model from the provided data by embedding time windows $\boldsymbol{y}_{t-k+1:t}$ of size k as internal state representation. Similar approaches have been used by auto-regressive HMMs (Shannon et al 2013). With longer data windows, the transitions become more and more Markov. How many observation each data window has to contain depends on two factors: on the dimensionality of the underlying system and on the signal-to-noise ratio of the measurements y_i.

5.2 The Subspace Kernel Kalman Filter

The subspace kernel Kalman filter (subKKF) is a natural extension of the KKF that leverages the subspace formulation of the conditional embedding operator presented in Equation 26 and its application to the framework for nonparametric inference as well as the subspace formulation of the kernel Kalman rule derived in Section 4.3.

In contrast to the KKF, we assume for the subKKF a data set of triples $\{(x_1, x'_1, y_1), \ldots, (x_m, x'_m, y_m)\}$, where x'_i is the successor state to x_i. The representation of the belief state changes from weight vector $\boldsymbol{m}_t$ and weight matrix $\boldsymbol{S}_t$ to the subspace projections of the embeddings $\boldsymbol{n}_t = \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{m}_t = \bar{\boldsymbol{K}}^\intercal \boldsymbol{m}_t$ and $\boldsymbol{P}_t = \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{S}_t \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x = \bar{\boldsymbol{K}}^\intercal \boldsymbol{S}_t \bar{\boldsymbol{K}}$, respectively. Additionally, both update procedures of the kernel Kalman filter, the transition update and the innovation update, have to be substituted by their subspace counterparts. The transition update is realized by the subspace kernel sum rule and the innovation update by the subspace kernel Kalman rule. The equations are depicted in Algorithm 2.

Since we represent the belief state as a projection into the subspace defined by $\boldsymbol{\Gamma}_x$, we can directly obtain the initial belief state by projecting the uniform embedding in the RKHS spanned by the samples from the initial distribution as

$$\boldsymbol{n}_0 = \boldsymbol{\Gamma}_x \boldsymbol{\Upsilon}_{x,0} \mathbf{1}_N \frac{1}{N} = \boldsymbol{K}_0^S \mathbf{1}_N \frac{1}{N}, \tag{78}$$

$$\boldsymbol{P}_0 = \frac{1}{N} \boldsymbol{\Gamma}_x \boldsymbol{\Upsilon}_{x,0} \boldsymbol{\Upsilon}_{x,0}^\intercal \boldsymbol{\Gamma}_x^\intercal = \frac{1}{N} \boldsymbol{K}_0^S \left(\boldsymbol{K}_0^S \right)^\intercal. \tag{79}$$

Here, $\boldsymbol{K}_0^S$ is the feature matrix of the reduced sample set and the samples from the initial state distribution. For the mapping back into the state space, we can similarly to the KKF define a subspace conditional operator as

$$C_{\text{pre}}^S := \boldsymbol{X} \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{\Gamma}_x^\intercal, \tag{80}$$

By applying this operator to mean map and covariance embedding, we obtain the mean and variance in state space from the subspace projections as

$$\boldsymbol{\eta}_t = \boldsymbol{X} \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{n}_t, \tag{81}$$

$$\boldsymbol{\Sigma}_t = \boldsymbol{X} \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{P}_t \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \bar{\boldsymbol{K}}^\intercal \boldsymbol{X}^\intercal. \tag{82}$$

A concise description of the subspace kernel Kalman filter can be found in Algorithm 2.

5.2.1 Selecting the Sample Set to Span the Subspace

To learn the subspace kernel Kalman filter models, we assume that we have a large data set of size m but we represent the belief states in a subspace spanned by only n samples, where $n \ll m$. A crucial problem is though how to select the subset of n samples from the full data set of m samples. We propose two approaches to address this problem which aim at different characteristics of the subset.

The first approach simply samples uniformly without replacement from the full data set. The result is a subset that resembles statistically the full data set, i.e., regions that have a high density in the full data set will have a high density in the subset and vice versa.

The second approach is a bit more sophisticated in its selection strategy. The goal is here to get an optimal coverage of the sample space in the subset according to a certain

measure. To do so, we select the first sample randomly from the full data set into the subset. Afterwards, we iteratively extend the subset by adding samples according to the following criterion: we compute the maximum kernel activation for each sample in the full data set with the samples in the current subset, then we extend the current subset by taking the sample from the full data set with the minimal maximum activation. We call this second strategy the *kernel activation heuristic*.

Algorithm 2: The Subspace Kernel Kalman Filter

input: triples $\{(x_1, x'_1, y_1), \ldots, (x_m, x'_m, y_m)\}$,
kernel functions k and g, regularization parameters λ and κ,
let $\boldsymbol{X}$ be the matrix of all x_i as columns, $\boldsymbol{X'}$ and $\boldsymbol{Y}$ analogously,
let $\boldsymbol{X}_0$ be the matrix of all data points used to compute the initial embeddings

select subset
$\boldsymbol{X}_S \sim$ random strategy, or $\boldsymbol{X}_S \sim$ min-max-activation strategy

compute kernel matrices
$\bar{\boldsymbol{K}} = k(\boldsymbol{X}, \boldsymbol{X}_S)$, $\bar{\boldsymbol{K}}' = k(\boldsymbol{X'}, \boldsymbol{X}_S)$, and $\boldsymbol{G} = g(\boldsymbol{Y}, \boldsymbol{Y})$

compute model matrices
$\boldsymbol{T}^S = \bar{\boldsymbol{K}}'^{\intercal}\bar{\boldsymbol{K}}\left(\bar{\boldsymbol{K}}^{\intercal}\bar{\boldsymbol{K}} + \lambda\boldsymbol{I}\right)^{-1}$ and $\boldsymbol{O}^S := \left(\bar{\boldsymbol{K}}^{\intercal}\bar{\boldsymbol{K}} + \lambda\boldsymbol{I}\right)^{-1}$

compute initial embeddings
kernel matrix with samples of the initial distribution: $\boldsymbol{K}_0^S = k(\boldsymbol{X}_S, \boldsymbol{X}_0)$
compute mean and variance over the columns: $\boldsymbol{m}_0 = \text{mean}\left(\boldsymbol{K}_0^S\right)$, $\boldsymbol{S}_0 = \text{var}\left(\boldsymbol{K}_0^S\right)$

loop
  if *new observation* $\boldsymbol{y}_t$ *available* **then**
    compute subspace kernel Kalman gain
    $\boldsymbol{Q}_t^S = \boldsymbol{P}_t^-\left(\boldsymbol{O}^S\right)^{\intercal}\left(\bar{\boldsymbol{K}}^{\intercal}\boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}_t^-\left(\boldsymbol{O}^S\right)^{\intercal} + \kappa\boldsymbol{I}\right)^{-1}\bar{\boldsymbol{K}}^{\intercal}$
    note that you can apply the matrix $\bar{\boldsymbol{K}}^{\intercal}$ *to the kernel matrix* $\boldsymbol{G}$ *in the innovation update already at learning time to increase computational efficiency.*
    innovation update
    $\boldsymbol{n}_t^+ = \boldsymbol{n}_t^- + \boldsymbol{Q}_t^S\left(\boldsymbol{g}_{:\boldsymbol{y}_t} - \boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{n}_t^-\right)$
    $\boldsymbol{P}_t^+ = \boldsymbol{P}_t^- - \boldsymbol{Q}_t^S\boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}_t^-$
  transition update
  $\boldsymbol{n}_{t+1}^- = \boldsymbol{T}^S\boldsymbol{n}_t^+, \quad \boldsymbol{P}_{t+1}^- = \boldsymbol{T}^S\boldsymbol{P}_t^+\left(\boldsymbol{T}^S\right)^{\intercal} + \boldsymbol{V}_S$
  project into state space
  $\boldsymbol{\eta}_t = \boldsymbol{X}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{n}_t, \quad \boldsymbol{\Sigma}_t = \boldsymbol{X}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}_t\boldsymbol{O}^S\bar{\boldsymbol{K}}^{\intercal}\boldsymbol{X}^{\intercal}$

5.3 Experimental Evaluation of the Kernel Kalman Filter

We evaluate the performance of the KKF and the subKKF on two experiments on simulated environments, a pendulum and a quad-link, and one experiment on real-world data from a human motion tracking data set (Wojtusch and von Stryk 2015). For all kernel based methods, we use the squared exponential kernel, where we choose the kernel bandwidths according to the *median trick* (Jaakkola et al 1999) and scale the median distances with a single optimized parameter.

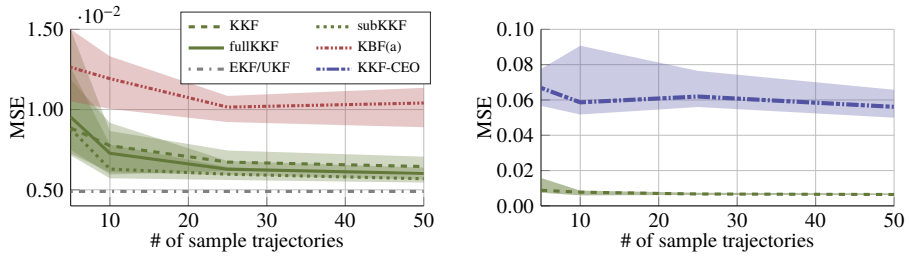


Fig. 3: Comparison of KKF to KBF(b), KKF-CEO, EKF and UKF. All kernel methods (except fullKKF) use kernel matrices of 100 samples. The subKKF method uses a subset of 100 samples and the whole dataset to learn the conditional operators. Depicted is the median MSE to the ground-truth of 20 trials with the [0.25 0.75] quantiles.

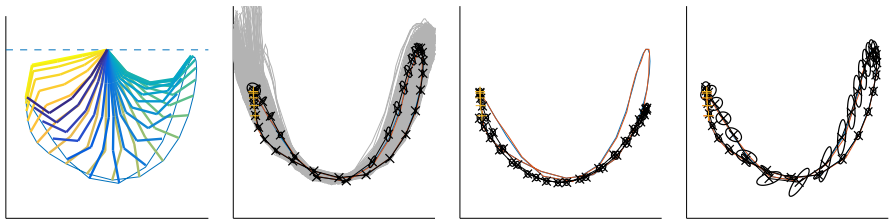
5.3.1 Pendulum

In this experiment, we used a simulated pendulum as system dynamics. It is uniformly initialized in the range $[0.1\pi, 0.4\pi]$ with a velocity sampled from the range $[-0.5\frac{\pi}{s}, 0.5\frac{\pi}{s}]$. The observations of the filters were the joint positions with additive Gaussian noise sampled from $\mathcal{N}(0, 0.01)$. We compared the KKF, the subspace KKF (subKKF) and the KKF learned with the full dataset (fullKKF) to version (a) of the kernel Bayes filter (KBF(a)) (Song et al 2013) (the other versions, KBF(b) and KBF(c), yielded worse results in this experiment) and the kernel Kalman filter with covariance embedding operator (KKF-CEO) (Zhu et al 2014), as well as to standard filtering approaches such as the EKF (Julier and Uhlmann 1997) and the UKF (Wan and Van Der Merwe 2000) (which require a model of the system dynamics). To learn the models, we simulated episodes with a length of 30 steps (3 sec). For the KKF and all KBF models, we use a kernel size of 100 samples, for the fullKKF, we use all available training samples and for the subKKF we use a set of 100 samples to span the subspace and the full data set to learn the operators. The samples are selected from the full data set using the kernel activation heuristic. The results are shown in Figure 3. The KKF and subKKF show clearly better results than all other non-parametric filtering methods and reach a performance level close to the EKF and UKF.

5.3.2 Quad-Link

In this experiment, we used a simulated 4-link pendulum where we observe the 2-D end-effector positions. The state of the pendulum consists of the four joint angles and joint velocities. We evaluate the prediction performance of the subKKF in comparison to the KKF-CEO, the EKF and the UKF. All other non-parametric filtering methods could not achieve a good performance or showed to be not feasible down due to the very high computation times. As the subKKF outperformed the KKF in the previous experiments and is also computationally much cheaper, we skip the comparison to the standard KKF in this and the subsequent experiments.

In a first qualitative evaluation, we compare the long-term prediction performance of the subKKF in comparison to the UKF, the EKF and the Monte-Carlo filter (MCF) as a baseline. This evaluation can be seen in Figure 4. The first five steps of of the end-effector trajectories were observed by the filters, the following 30 steps were predicted. The UKF is not able to



(a) anim. QL (b) MCF (c) UKF (d) subKKF

Fig. 4: Example trajectory of the quad-link end-effector. The filter outputs in black, where the ellipses enclose 90% of the probability mass. All filters were updated with the first five measurements (yellow marks) and predicted the following 30 steps. Figure (a) is an animation of the trajectory.

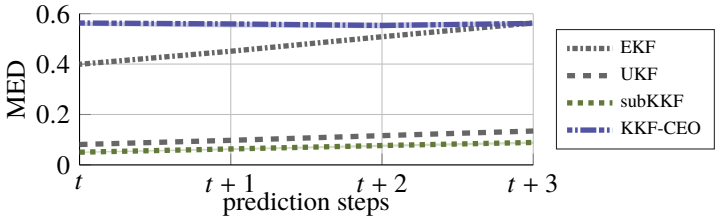


Fig. 5: 1, 2 and 3 step prediction performances in mean euclidean distances (MED) to the true end-effector positions of the quad-link.

predict the movements of the quad-link end-effector due to the high non-linearity, while the subKKF is able to predict the whole trajectory.

We also compared the 1, 2 and 3-step prediction performance of the subKKF to the KKF-CEO, EKF and UKF (Fig. 5). The KKF-CEO provides poor results already for the filtering task. The EKF performs equally bad, since the observation model is highly non-linear. The UKF already yields a much better performance as it does not suffer from the linearizion of the system dynamics. The subKKF outperformed the UKF.

5.3.3 Human Motion Data

The human motion dynamics (HuMoD) database by Wojtusch and von Stryk (2015) consists of the datasets of several motions executed by two subjects. All datasets contain the recordings from a motion capture system with 36 markers as well as the recordings of the electrical activity of 14 muscles in the legs. Additionally, data from the treadmill such as ground reaction forces and velocities are available. The x-, y-, and z-locations of the markers were recorded at 500Hz, the muscle activities at 2000Hz, and the data from the treadmill at 1000Hz. Furthermore, the database contains joint positions and joint trajectories derived from the marker positions via a kinematic model of the human body. In our experiments, we use the marker locations, the derived locations of the joints and the muscle activities. We subsample all data to a common frame rate of 50Hz and transpose the x- and z-position of all markers such that the T12-marker (marker at the 12th thoracic vertebra) has ($x = 0, z = 0$) in all frames. Note that in the HuMoD database, the x-axis points in the motion direction (i.e.,

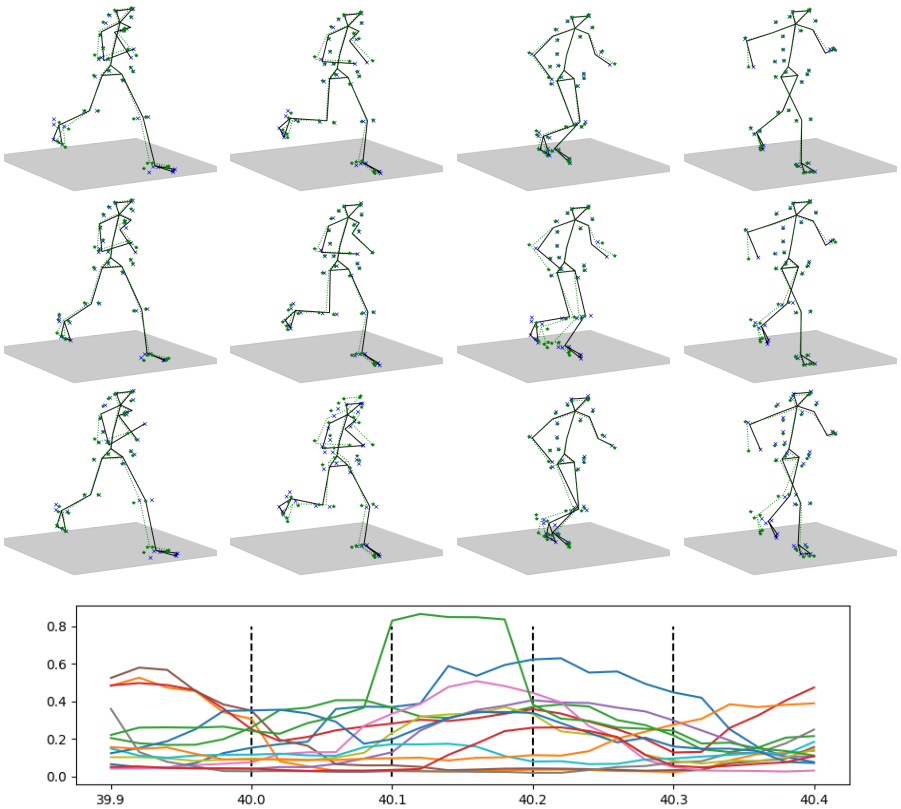


Fig. 6: Example sequence of 4 postures and the measured muscle activities. The marker and skeleton in green depict the ground-truth, the estimates from the models are depicted in black/blue. The learned models estimate the marker and joint positions from the muscle activities. The first row shows the estimated positions from the subKKF, the second row shows the estimated positions from the subKBF and the third row shows the estimated positions from a sparse GP. For all three models, we used a sample set of 2000 samples and a sparse subset of 500 samples.

along the treadmill), the y-axis points upwards and the z-axis forms a right-hand coordinate system towards the right side of the treadmill. We used walking motions at 1.0m/s, 1.5m/s, 2.0m/s, and running motions at 2.0m/s, 3.0m/s, and 4m/s, captured from one subject. For evaluating the trained model, we used a test data-set in which the subject transitions linearly from 0m/s up to 4m/s and back to 0m/s.

In the experiment, we compare the performance of subKKF, subKBF, and sparse Gaussian process (SGP) in restoring the marker and joint positions from the muscle activities. We learn all three models using the marker and joint positions as state variables (or outputs) x_i and the muscle activities as observations (or inputs) y_i. We use a set of 2000 samples to learn the kernel matrices and a subset of 500 samples to define the subspace (or as inducing inputs). For subKKF and subKBF, we used a window size of 2. While we could easily carry out the optimization of the parameters for the subKKF and for the SGP, the optimization of the parameters for the subKBF was not feasible in a considerable amount of time.

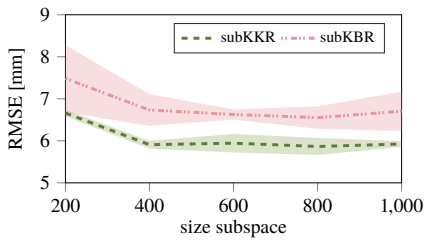


Fig. 7: Performance of the subKKF and the subKBF on the HuMoD transition data for different sizes of the subspace.

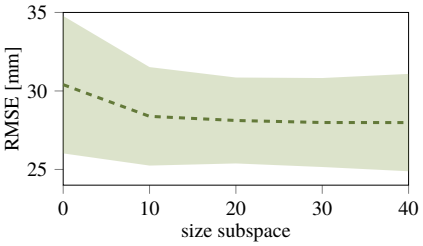


Fig. 8: Performance of the subKKF on HuMoD test sequences after 0, 10, 20, 30 and 40 iterations of the CMA-ES optimizer.

#	200	400	600	800	1000
subKKF	**6.66**	**5.91**	**5.94**	**5.87**	**5.92**
subKBF	7.48	6.73	6.63	6.55	6.70
SGP	76.33	70.15	68.46	63.13	58.97

subKKF	**0.61±0.10 s**	**1.71±0.17 s**	**3.84±0.40 s**	**7.21±0.85 s**	**11.72±0.41 s**
subKBF	59±1.3 s	170±11.3 s	352±15 s	620±15 s	927±174 s

Table 2: Top: performance of subKKF, subKBF, and SGP for the HuMoD transition data for different sizes of the subspace. Bottom: time consumptions of subKKF and subKBF for filtering 100 test sequences of length 50 from the HuMoD data set.

Figure 6 depicts marker and joint positions of four exemplary postures together with the muscle activities during that period of time. The locations of the exemplary postures in the time linen are depicted by vertical lines in the plot of the muscle activities. While this is only a qualitative example, it depicts how the subKKF outperforms the subKBF and the SGP in restoring the positions of the markers and the joints.

We compare the performance of subKKF, subKBF and SGP for different sizes of the subspace (inducing inputs). Figure 7 depicts the performance of subKKF and subKBF. The results of the SGP can be seen in Table 2 which are clearly worse in comparison to the filtering approaches which take the temporal correlation of the data into account. Furthermore, Table 2 also depicts the time consumption of subKKF and subKBF for the evaluation of the transition data in the HuMoD data base. The gain in efficiency of the subKKF over the subKBF which is around the factor 100 can be seen clearly. In Figure 8, we depict the performance gain of the subKKF over the number of iterations of the CMA-ES optimizer. We see that in this case, the first 10 iterations yield a bigger jump in performance than the following 40 steps. However, from our experience, this is very specific to the problem and the initial setting of the parameters, which were in this experiment already very close to the optimal parameters.

5.4 The Kernel Forward-Backward Smoother

Smoothing is in contrast to filtering a post-processing routine. While filtering refers to a routine where the current state is estimated recursively from all past observations, smoothing computes the best state estimates given all available observations from the past and the

future. Hence, for a given time series of observations $[y_1, \ldots, y_T]$, we want to obtain the belief $p(x_t|y_1, \ldots, y_T)$ for all $1 \leq t \leq T$.

One well-known and simple approach to smoothing is the forward-backward smoother. During a forward pass the standard filtering algorithm is applied to the observations. Afterwards, on the backward pass, an inverse filter is applied to the same time series of observations. Finally the filter estimates of forward and backward pass are combined into the smoothed estimates. Since the information from the observation should be incorporated only once into the smoothed estimate, we need to combine the posterior estimate of the forward pass with the prior estimate of the backward pass (or vice versa). The problems of this routine are usually that it requires an inverse model of the underlying system for the backward pass and an initialization of the final state. If we assume that the system has a finite state space, these issues are however not applicable for the KKF as we can learn the models in the feature space from data.

5.4.1 Computing the Smoothed Belief State as a Weighted Average

Assuming that we have the a-posteriori belief states from the forward pass and the a-priori belief states from the backward pass as

$$\left\{\left(\mu_{f,1}^{+}, C_{f,1}^{+}\right), \ldots, \left(\mu_{f,T}^{+}, C_{f,T}^{+}\right)\right\} \qquad \text{and} \qquad \left\{\left(\mu_{b,1}^{-}, C_{b,1}^{-}\right), \ldots, \left(\mu_{b,T}^{-}, C_{b,T}^{-}\right)\right\}, \tag{83}$$

respectively, we can combine the mean maps into a smoothed belief state as the weighted average

$$\mu_{s,t} = \mathcal{Z}_{f,t}\mu_{f,t}^{+} + \mathcal{Z}_{b,t}\mu_{b,t}^{-}. \tag{84}$$

Since both, the estimator from the forward pass and the esitmator from the backward pass, are unbiased, the weighting operators $\mathcal{Z}_{f,t}$ and $\mathcal{Z}_{b,t}$ need to satisfy $\mathcal{I} = \mathcal{Z}_{f,t} + \mathcal{Z}_{b,t}$, i.e.,

$$\mathbb{E}\left[\boldsymbol{\varphi}(X_t) - \mu_t^s\right] \stackrel{!}{=} 0 \tag{85}$$

$$\mathbb{E}\left[\boldsymbol{\varphi}(X_t) - \mathcal{Z}_{f,t}\mu_t^{f^+} + \mathcal{Z}_{b,t}\mu_t^{b^-}\right] \stackrel{!}{=} 0 \tag{86}$$

$$\mathbb{E}\left[\boldsymbol{\varphi}(X_t)\right] - \mathcal{Z}_{f,t}\mathbb{E}\left[\mu_t^{f^+}\right] + \mathcal{Z}_{b,t}\mathbb{E}\left[\mu_t^{b^-}\right] \stackrel{!}{=} 0 \tag{87}$$

$$\mu_t - \left(\mathcal{Z}_{f,t} + \mathcal{Z}_{b,t}\right)\mu_t \stackrel{!}{=} 0 \tag{88}$$

$$\Rightarrow \quad \mathcal{Z}_{f,t} + \mathcal{Z}_{b,t} = \mathcal{I} \tag{89}$$

Thus, the weighting operators can be expressed by each other as $\mathcal{Z}_{b,t} = \mathcal{I} - \mathcal{Z}_{f,t}$ and vice versa. We substitute this representation back into the smoothing update in Equation 84 to obtain

$$\mu_{s,t} = \mathcal{Z}_{f,t}\mu_{f,t}^{+} + \left(\mathcal{I} - \mathcal{Z}_{f,t}\right)\mu_{b,t}^{-}. \tag{90}$$

5.4.2 Finding the Optimal Weighting Operators

We obtain the optimal weighting operators by minimizing the squared error of the smoothing mean map which is equivalent to minimizing the trace of the smoothed covariance embedding operator $C_{s,t}$, i.e.,

$$\mathbb{E}\left[\left(\boldsymbol{\varphi}(X_t) - \mu_t^s\right)^{\mathsf{T}}\left(\boldsymbol{\varphi}(X_t) - \mu_t^s\right)\right] = \mathbb{E}\left[\mathrm{Tr}\left(\boldsymbol{\varphi}(X_t) - \mu_t^s\right)\left(\boldsymbol{\varphi}(X_t) - \mu_t^s\right)^{\mathsf{T}}\right] = \mathrm{Tr}\, C_{s,t}. \tag{91}$$

We can then use Equation 90 to rewrite the covariance opartor as

$$C_{s,t} = \mathbb{E}\left[\left(\boldsymbol{\varphi}(X_t) - \mathcal{Z}_{f,t}\mu^+_{f,t} + \left(\mathcal{I} - \mathcal{Z}_{f,t}\right)\mu^-_{b,t}\right)\left(\dots\right)^\intercal\right] \tag{92}$$

$$= \mathbb{E}\left[\left(\mathcal{Z}_{f,t}(\epsilon_f - \epsilon_b) + \epsilon_b\right)\left(\dots\right)^\intercal\right], \tag{93}$$

where $\epsilon_f = \boldsymbol{\varphi}(x_t) - \mu^+_{f,t}$ is the error of the a-posteriori estimate of the forward pass and $\epsilon_b = \boldsymbol{\varphi}(x_t) - \mu^-_{b,t}$ is the error of the a-priori estimate of the backward pass and where we used the relation $\boldsymbol{\varphi}(X_t) = \mathcal{I}\boldsymbol{\varphi}(X_t) = (\mathcal{Z}_{f,t} + \mathcal{Z}_{b,t})\boldsymbol{\varphi}(X_t) = (\mathcal{Z}_{f,t} + (\mathcal{I} - \mathcal{Z}_{f,t}))\boldsymbol{\varphi}(X_t)$. By expanding the square and since the cross-covariances of the errors from forward and the backward pass are zero (i.e., $\mathbb{E}[\epsilon_f \epsilon_b^\intercal] = 0$), we arrive at

$$C_{s,t} = \mathbb{E}\left[\mathcal{Z}_{f,t}\left(\epsilon_f\epsilon_f^\intercal + \epsilon_b\epsilon_b^\intercal\right)\mathcal{Z}_{f,t}^\intercal - \mathcal{Z}_{f,t}\epsilon_b\epsilon_b^\intercal - \epsilon_b\epsilon_b^\intercal\mathcal{Z}_{f,t}^\intercal + \epsilon_b\epsilon_b^\intercal\right]. \tag{94}$$

Lastly, we can take the derivative and set it to zero to obtain the optimal $\mathcal{Z}_{f,t}$ as

$$0 \overset{!}{=} \frac{\partial \operatorname{Tr} C_{s,t}}{\partial \mathcal{Z}_{f,t}} = 2\mathbb{E}\left[\mathcal{Z}_{f,t}\left(\epsilon_f\epsilon_f^\intercal + \epsilon_b\epsilon_b^\intercal\right) - \epsilon_b\epsilon_b^\intercal\right] \tag{95}$$

$$0 \overset{!}{=} \mathcal{Z}_{f,t}\left(\mathbb{E}\left[\epsilon_f\epsilon_f^\intercal\right] + \mathbb{E}\left[\epsilon_b\epsilon_b^\intercal\right]\right) - \mathbb{E}\left[\epsilon_b\epsilon_b^\intercal\right] \tag{96}$$

$$0 \overset{!}{=} \mathcal{Z}_{f,t}\left(C^+_{f,t} + C^-_{b,t}\right) - C^-_{b,t} \tag{97}$$

$$\mathcal{Z}_{f,t} = C^-_{b,t}\left(C^+_{f,t} + C^-_{b,t}\right)^{-1}. \tag{98}$$

From the condition on the weighting operators stated in Equation 89, it furthermore follows that $\mathcal{Z}_{b,t} = C^-_{f,t}\left(C^+_{f,t} + C^-_{b,t}\right)^{-1}$.

5.4.3 Smoothing the Covariance Embedding Operator

Taking the representation of the smoothed covariance in Equation 94 and substituting the covariance operators and the optimal weighting operator $\mathcal{Z}_{f,t}$ gives us the following smoothed covariance operator

$$C_{s,t} = C^-_{b,t} - C^-_{b,t}\left(C^+_{f,t} + C^-_{b,t}\right)^{-1}C^-_{b,t} \tag{99}$$

$$= C^-_{b,t} - \left(\mathcal{I} - C^+_{f,t}\left(C^+_{f,t} + C^-_{b,t}\right)^{-1}\right)C^-_{b,t} \tag{100}$$

$$= C^+_{f,t}\left(C^+_{f,t} + C^-_{b,t}\right)^{-1}C^-_{b,t} \tag{101}$$

From the optimal solution of the weighting operator, we can now see that the smoothing update of the covariance embedding operator can be expressed as

$$C_{s,t} = \mathcal{Z}_{b,t}C^-_{b,t} = \mathcal{Z}_{f,t}C^+_{f,t} \tag{102}$$

In the following section, we will show how the smoothing update can be expressed with finite samples using vector/matrix operations.

5.4.4 The Empirical Kernel Forward-Backward Smoother

We assume that we are given the weight vectors and weight matrices from the forward and the backward pass as $\{(\boldsymbol{m}^+_{f,1}, \boldsymbol{S}^+_{f,1}), \ldots, (\boldsymbol{m}^+_{f,T}, \boldsymbol{S}^+_{f,T})\}$ and $\{(\boldsymbol{m}^-_{b,1}, \boldsymbol{S}^-_{b,1}), \ldots, (\boldsymbol{m}^-_{b,T}, \boldsymbol{S}^-_{b,T})\}$, respectively. Since the weighting operator $\mathcal{Z}_{b,t}$ can be expressed by $\mathcal{Z}_{f,t}$ and vice versa, we only need to compute one of the weighting operators which we choose to be $\mathcal{Z}_{f,t}$. We add the identity operator with a small scalar γ in the inverse to improve the numerical stability and obtain

$$\mathcal{Z}_{f,t} = C^-_{b,t}\left(C^+_{f,t} + C^-_{b,t} + \gamma\mathcal{I}\right)^{-1} \tag{103}$$

$$= \boldsymbol{\Upsilon}_x \boldsymbol{S}^-_{b,t} \boldsymbol{\Upsilon}_x^\intercal \left(\boldsymbol{\Upsilon}_x\left(\boldsymbol{S}^+_{f,t} + \boldsymbol{S}^-_{b,t}\right)\boldsymbol{\Upsilon}_x^\intercal + \gamma\mathcal{I}\right)^{-1} \tag{104}$$

$$= \boldsymbol{\Upsilon}_x \boldsymbol{S}^-_{b,t} \left(\boldsymbol{\Upsilon}_x^\intercal\boldsymbol{\Upsilon}_x\left(\boldsymbol{S}^+_{f,t} + \boldsymbol{S}^-_{b,t}\right) + \gamma\boldsymbol{I}\right)^{-1} \boldsymbol{\Upsilon}_x^\intercal \tag{105}$$

$$= \boldsymbol{\Upsilon}_x \underbrace{\boldsymbol{S}^-_{b,t}\left(\boldsymbol{K}\left(\boldsymbol{S}^+_{f,t} + \boldsymbol{S}^-_{b,t}\right) + \gamma\boldsymbol{I}\right)^{-1}}_{\boldsymbol{Z}_{f,t}} \boldsymbol{\Upsilon}_x^\intercal, \tag{106}$$

where we used again the matrix identity $\boldsymbol{A}(\boldsymbol{B}\boldsymbol{A} + \boldsymbol{I})^{-1} = (\boldsymbol{A}\boldsymbol{B} + \boldsymbol{I})^{-1}\boldsymbol{A}$ and defined the finite weighting matrix $\boldsymbol{Z}_{f,t}$. With this weighting matrix we can now combine the mean maps of the forward and the backward pass as

$$\mu_{s,t} = \mathcal{Z}_{f,t}\mu^+_{f,t} + \left(\mathcal{I} - \mathcal{Z}_{f,t}\right)\mu^-_{b,t} \tag{107}$$

$$= \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{\Upsilon}_x^\intercal \mu^+_{f,t} + \left(\mathcal{I} - \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{\Upsilon}_x^\intercal\right)\mu^-_{b,t} \tag{108}$$

$$= \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{K} \boldsymbol{m}^+_{f,t} + \boldsymbol{\Upsilon}_x \boldsymbol{m}^-_{b,t} - \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{K} \boldsymbol{m}^-_{b,t} \tag{109}$$

$$\boldsymbol{\Upsilon}_x \boldsymbol{m}^s_t = \boldsymbol{\Upsilon}_x\left(\boldsymbol{m}^-_{b,t} + \boldsymbol{Z}_{f,t}\boldsymbol{K}\left(\boldsymbol{m}^+_{f,t} - \boldsymbol{m}^-_{b,t}\right)\right). \tag{110}$$

And similarly we can obtain the smoothed estimate of the covariance operator as

$$C^s_t = \mathcal{Z}_{f,t} C^+_{f,t} \tag{111}$$

$$= \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{S}^+_{f,t} \boldsymbol{\Upsilon}_x^\intercal \tag{112}$$

$$\boldsymbol{\Upsilon}_x \boldsymbol{S}^s_t \boldsymbol{\Upsilon}_x^\intercal = \boldsymbol{\Upsilon}_x \boldsymbol{Z}_{f,t} \boldsymbol{K} \boldsymbol{S}^+_{f,t} \boldsymbol{\Upsilon}_x^\intercal \tag{113}$$

A concise description of the kernel forward-backward smoothing algorithm can be found in Algorithm 3.

5.4.5 Initialization of the Backward Kernel Kalman Filter

A critical aspect of the classical forward-backward smoothing algorithm is the initialization of the belief state for the backward pass. Often the distribution over the initial state is well known but not a distribution over the terminal state, where it is often not even clear how a terminal state is defined. For the backward kernel Kalman filter, two approaches can be used to initialize the belief state. The first approach assumes that we have multiple episodes in the training data, where each episode terminates in a terminal state of the system. We can then compute the initialization for the backward pass analogously to the initialization for the KKF described in Section 5.1.1. The second approach simply assumes that the system has a stationary distribution which is covered by the training data. The initialization is then the embedding of the distribution over all the samples in the training set.

Algorithm 3: The Kernel Forward-Backward Smoother

input: triples $\{(\tilde{x}_1, x_1, x'_1, y_1), \ldots, (\tilde{x}_m, x_m, x'_m, y_m)\}$,
kernel functions k and g, regularization parameters λ and κ,
let $\boldsymbol{X}$ be the matrix of all x_i as columns, $\tilde{\boldsymbol{X}}$, $\boldsymbol{X'}$ and $\boldsymbol{Y}$ analogously,
let $\boldsymbol{X}_0$ be the matrix of all data points used to initialize the forward pass
let $\boldsymbol{X}_T$ be the matrix of all data points used to initialize the backward pass

learn the forward filter
See Algorithm 1 with data matrices $\boldsymbol{X}_0$, $\tilde{\boldsymbol{X}}$, $\boldsymbol{X}$, and $\boldsymbol{Y}$.

learn the backward filter
See Algorithm 1 with data matrices $\boldsymbol{X}_T$, $\boldsymbol{X}$, $\boldsymbol{X'}$, and $\boldsymbol{Y}$, note that you need to learn the transition model from $\boldsymbol{X'}$ to $\boldsymbol{X}$.

apply forward filter
Compute filtered estimates $\left\{\left(\boldsymbol{m}^+_{f,1}, \boldsymbol{S}^+_{f,1}\right), \ldots, \left(\boldsymbol{m}^+_{f,T}, \boldsymbol{S}^+_{f,T}\right)\right\}$

apply backward filter and compute smoothed estimates
loop
  if *observation* $\boldsymbol{y}_t$ *available* **then**
    compute kernel Kalman gain
    $\boldsymbol{Q}_{b,t} = \boldsymbol{S}^-_{b,t}\boldsymbol{O}^\intercal(\boldsymbol{G}\left(\boldsymbol{O}\boldsymbol{S}^-_{b,t}\boldsymbol{O}^\intercal + \boldsymbol{R}_b\right) + \kappa\boldsymbol{I})^{-1}$
    innovation update
    $\boldsymbol{m}^+_{b,t} = \boldsymbol{m}^-_{b,t} + \boldsymbol{Q}_{b,t}(\boldsymbol{g}_{:y_t} - \boldsymbol{G}\boldsymbol{O}\boldsymbol{m}^-_t)$
    $\boldsymbol{S}^+_{b,t} = \boldsymbol{S}^-_{b,t} - \boldsymbol{Q}_{b,t}\boldsymbol{G}\boldsymbol{O}\boldsymbol{S}^-_{b,t}$
  transition update
  $\boldsymbol{m}^-_{b,t-1} = \boldsymbol{T}_b\boldsymbol{m}^+_{b,t}, \quad \boldsymbol{S}^-_{b,t-1} = \boldsymbol{T}_b\boldsymbol{S}^+_{b,t}\boldsymbol{T}_b^\intercal + \boldsymbol{V}_b$
  compute smoothed estimate
  $\boldsymbol{Z}_{f,t} = \boldsymbol{S}^-_{b,t}\left(\boldsymbol{K}\left(\boldsymbol{S}^+_{f,t} + \boldsymbol{S}^-_{b,t}\right) + \gamma\boldsymbol{I}\right)^{-1}$
  $\boldsymbol{m}^s_t = \boldsymbol{m}^-_{b,t} + \boldsymbol{Z}_{f,t}\boldsymbol{K}\left(\boldsymbol{m}^+_{f,t} - \boldsymbol{m}^-_{b,t}\right)$
  $\boldsymbol{S}^s_t = \boldsymbol{Z}_{f,t}\boldsymbol{K}\boldsymbol{S}^+_{f,t}$
  project into state space
  $\boldsymbol{\eta}^s_t = \boldsymbol{X}\boldsymbol{O}\boldsymbol{m}^s_t, \quad \boldsymbol{\Sigma}^s_t = \boldsymbol{X}\boldsymbol{O}\boldsymbol{S}^s_t\boldsymbol{O}^\intercal\boldsymbol{X}^\intercal$

5.5 The Subspace Kernel Forward-Backward Smoother

If we use the subspace kernel Kalman filter to perform the forward and the backward pass, we obtain as outcome the subspace projections $\boldsymbol{n}_t$ of the mean map and $\boldsymbol{P}_t$ covariance embedding instead of the weight vectors $\boldsymbol{m}_t$ and weight matrices $\boldsymbol{S}_t$, respectively. To perform smoothing on these subspace projections, we need to find the weighting matrices for the smoothing update analog to Equation 90. Though, as the representation is already in a finite domain, we can directly apply the optimal solution found in Equation 98 to the subspace projections of the covariance operator. Hence, the weighting matrix for the subspace kernel forward-backward smoother (subKFBS) becomes

$$\boldsymbol{Z}^S_{f,t} = \boldsymbol{P}^-_{b,t}\left(\boldsymbol{P}^+_{f,t} + \boldsymbol{P}^-_{b,t}\right)^{-1} \tag{114}$$

From here, we can obtain the equations for the smoothing update of the subspace kernel forward-backward smoother easily by

$$\boldsymbol{n}_{s,t} = \boldsymbol{Z}^S_{f,t}\boldsymbol{n}^+_{f,t} + \left(\boldsymbol{I} - \boldsymbol{Z}^S_{f,t}\right)\boldsymbol{n}^-_{b,t}, \qquad \text{and} \tag{115}$$

$$\boldsymbol{P}_{s,t} = \boldsymbol{Z}^S_{f,t}\boldsymbol{P}^+_{f,t}. \tag{116}$$

Algorithm 4 gives a compact description of the subKFBS.

Algorithm 4: The Subspace Kernel Forward-Backward Smoother

input: triples $\{(\tilde{x}_1, x_1, x'_1, y_1), \ldots, (\tilde{x}_m, x_m, x'_m, y_m)\}$,
kernel functions k and g, regularization parameters λ and κ,
let $\boldsymbol{X}$ be the matrix of all x_i as columns, $\tilde{\boldsymbol{X}}$, $\boldsymbol{X'}$ and $\boldsymbol{Y}$ analogously,
let $\boldsymbol{X}_0$ be the matrix of all data points used to initialize the forward pass
let $\boldsymbol{X}_T$ be the matrix of all data points used to initialize the backward pass

learn forward and backward filter
See Algorithm 1 with data matrices $\boldsymbol{X}_0$, $\boldsymbol{X}$, $\boldsymbol{X'}$, and $\boldsymbol{Y}$ for the forward filter and data matrices $\tilde{\boldsymbol{X}}$, $\boldsymbol{X}$, and $\boldsymbol{Y}$ for the backward filter. Note that you need to learn the transition model for the backward filter from $\boldsymbol{X}$ to $\tilde{\boldsymbol{X}}$.

apply forward filter
Compute filtered estimates $\left\{\left(\boldsymbol{n}^+_{f,1}, \boldsymbol{P}^+_{f,1}\right), \ldots, \left(\boldsymbol{n}^+_{f,T}, \boldsymbol{P}^+_{f,T}\right)\right\}$

apply backward filter and compute smoothed estimates
loop
  if *observation* $\mathbf{y}_t$ *available* **then**
    compute subspace kernel Kalman gain
    $$\boldsymbol{Q}^S_t = \boldsymbol{P}^-_{b,t}\left(\boldsymbol{O}^S\right)^\intercal\left(\bar{\boldsymbol{K}}^\intercal\boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}^-_{b,t}\left(\boldsymbol{O}^S\right)^\intercal + \kappa\boldsymbol{I}\right)^{-1}\bar{\boldsymbol{K}}^\intercal$$
    innovation update
    $$\boldsymbol{n}^+_{b,t} = \boldsymbol{n}^-_{b,t} + \boldsymbol{Q}^S_t\left(\boldsymbol{g}_{:\mathbf{y}_t} - \boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{n}^-_{b,t}\right), \quad \boldsymbol{P}^+_{b,t} = \boldsymbol{P}^-_{b,t} - \boldsymbol{Q}^S_t\boldsymbol{G}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}^-_{b,t}$$
  transition update
  $$\boldsymbol{n}^-_{b,t-1} = \boldsymbol{T}^S\boldsymbol{n}^+_{b,t}, \quad \boldsymbol{P}^-_{b,t-1} = \boldsymbol{T}^S\boldsymbol{P}^+_{b,t}\left(\boldsymbol{T}^S\right)^\intercal + \boldsymbol{V}^S$$
  compute smoothed estimate
  $$\boldsymbol{Z}^S_{f,t} = \boldsymbol{P}^-_{b,t}\left(\boldsymbol{P}^+_{f,t} + \boldsymbol{P}^-_{b,t}\right)^{-1}$$
  $$\boldsymbol{n}_{s,t} = \boldsymbol{Z}^S_{f,t}\boldsymbol{n}^+_{f,t} + \left(\boldsymbol{I} - \boldsymbol{Z}^S_{f,t}\right)\boldsymbol{n}^-_{b,t}, \quad \boldsymbol{P}_{s,t} = \boldsymbol{Z}^S_{f,t}\boldsymbol{P}^+_{f,t}$$
  project into state space
  $$\boldsymbol{\eta}_{s,t} = \boldsymbol{X}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{n}_{s,t}, \quad \boldsymbol{\Sigma}_{s,t} = \boldsymbol{X}\bar{\boldsymbol{K}}\boldsymbol{O}^S\boldsymbol{P}_{s,t}\boldsymbol{O}^S\bar{\boldsymbol{K}}^\intercal\boldsymbol{X}^\intercal$$

5.6 Experimental Evaluation of the Kernel Forward Backward Smoother

We evaluate the kernel forward backward smoother with two experiments. In the first experiment on data from a simulated pendulum, we show the performance gain of the KFBS over the KKF. In the second experiment, we apply the KFBS on data of a table tennis ball recorded with a camera-based tracking system and show how the KFBS and subKFBS are able to restore the full trajectory of the ball while only having observations at the first four and at the last time step.

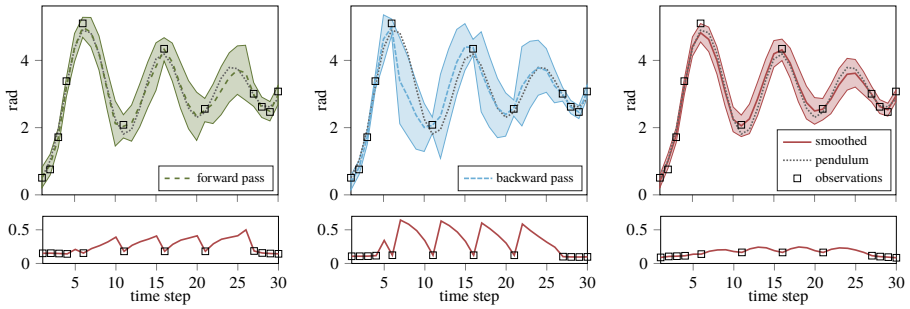


Fig. 9: Qualitative comparison of the forward and the backward pass to the smoothed estimates of the KFBS on a simulated pendulum. The upper plots show the mean and variance output of the filter/smoother, the lower plots show the profiles of the standard deviation. While the forward pass already yields good estimates in the first half of the time series, the smoother incorporates the good estimates from the backward pass in the second half and outperforms the filters. In addition, the smoother yields a more confident about its estimate.

5.6.1 Pendulum

We simulate a pendulum similar to the one from Section 5.3.1, however we initialize the pendulum in the range $[-0.25\pi, 0.25\pi]$ and with a angular velocity sampled from the range $[-2\frac{\pi}{s}, 2\frac{\pi}{s}]$. During the simulation, we apply Gaussian process noise with $\sigma = 0.01$ and as observations we use the angular displacement with Gaussian observation noise with $\sigma = 0.2$. To learn the KFBS models, we sample 100 episodes with each 30 steps where a step corresponds to 0.1 second. The training samples are 200 windows of four observations, which we select by the kernel activation heuristic explained in Section 5.2.1. To find the optimal parameters, we apply CMA-ES (Hansen 2006) where we use the negative log-likelihood of the ground-truth to the smoothed estimate as optimality criterion. During the optimization, we use a test-dataset of 10 episodes, where we still observe at each time step. Later, we evaluate the smoothing performance on an evaluation-dataset where we do not observe at each time step but only at $t = [1 - 4, 6, 11, 16, 21, 27 - 30]$. This optimization procedure yielded better results than directly optimizing with only partial observations.

In Figure 9, we show a qualitative comparison of the forward and the backward pass to the smoother. The results are as expected: the forward pass yields better results in the first half of the episode, and the backward pass yields better results in the second half. The smoother combines both estimates and outperforms the filter results. The smoothing can also be observed in the profiles of the standard deviation. While the variance from the filters increases at each time step without observation until the next measurement, the variance of the smoother is much smaller and only rises slightly between the observations.

In Figure 10, we compare the performance of a standard KKF to the KFBS and the subKFBS for different kernel sizes on the same state estimation task for a simulated pendulum. The subKFBS has been learned with 300 samples in the full data set. Depicted are the median and the $[0.15, 0.85]$-quantiles of the MSE over 20 repetitions. The KFBS and the subKFBS clearly outperform the KKF for small kernel sizes (50, 100) and also yield better results for larger kernel sizes(150, 200). The subKFBS yields slightly better results than the KFBS. In

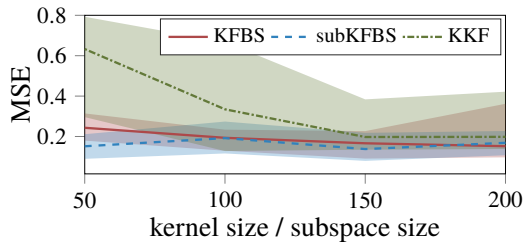


Fig. 10: The KFBS and the sub KFBS outperform the KKF clearly for small kernel sizes but also with more samples in the gram matrices. The task was to estimate the state of a pendulum from noisy partial observations. Depicted are the median and the $[0.15, 0.85]$-quantiles of the MSE over 20 repetitions.

addition, we see from the quantiles of the MSE that the KFBS and the subKFBS have a more stable behaviour in the optimization process than the KKF.

5.6.2 Tabletennis

In a second experiment, we perform smoothing on observations of a table tennis ball (Gomez-Gonzalez et al 2016). The data set contains 54 trajectories of a table tennis ball tracked with a camera system, where each trajectory contains 51 observations which are recorded with a frequency of 100 Hz. We train the subKFBS with 34 trajectories, use 10 trajectories for optimizing the parameters using CMA-ES (Hansen 2006). The remaining 10 trajectories are used for evaluating the results. For the smoothing task, the ball has been observed at the first five time steps and then again at the last time step.

Figure 11 shows qualitative examples of smoothed trajectories using the subKFBS in comparison the output of the subKKF. Here, we used data windows of size 4 and learned the models with 300 samples in the training data set and 100 samples in the subset. We optimized the regularization parameters and all bandwidths with CMA-ES (Hansen 2006). The plot shows how the subKFBS can estimate accurately the path of the ball only from observations at the beginning and at the end of the trajectory, while the subKKF diverges from the actual trajectory over time. In addition, the plot also depcits how the subKFBS, which is only learned from noisy observations, returns a trajectory that is more smooth than the noisy data recorded by a camera-based tracking system.

We also compare the KFBS to the subKFBS for different kernel sizes with the same smoothing task on recorded table tennis ball data. Figure 12 shows a comparison of the MSE, depicting the median and the $[0.05, 0.95]$-quantiles over 20 repetitions. The KFBS has been learned with a varying kernel size of 50, 100, 150, and 200 samples. The subKFBS uses the same number of samples to span the subspace but learns the models always with 400 samples in the full training set. While the subKFBS outperforms the KFBS for all kernel sizes, the KFBS achieves a similar performance to the KFBS when learned with 200 samples.

6 Conclusion & Future Work

In this paper, we have presented the kernel Kalman rule (KKR) as an alternative to the kernel Bayes' rule (KBR) in the framework for nonparametric inference Song et al (2013). In contrast to the KBR, the KKR is computationally more efficient, numerically more stable and follows from a clear optimization objective. We have further combined the KKR as Bayesian update with the kernel sum rule to formulate the kernel Kalman filter (KKF). The kernel

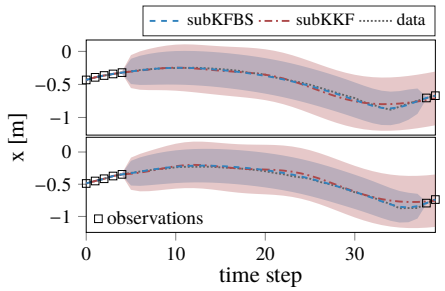


Fig. 11: In comparison to the KKF, the KFBS is able to reconstruct the trajectories of a table tennis ball from observations at the first five and at the last two time steps. The plot shows the z-coordinate of two trajectories of a table tennis ball recorded with a camera-based tracking system. We learned the subKFBS/subKKF with 100 samples in the subspace and 300 points in the training set.

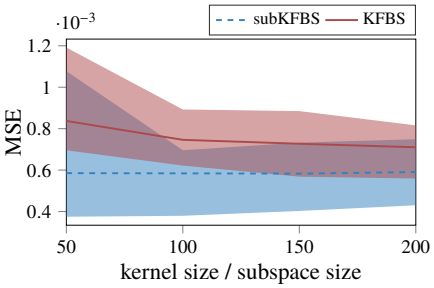


Fig. 12: The subKFBS performs better than the KFBS in the table tennis ball smoothing task. This difference in the MSE between the estimates and the noisy recorded data is more prevalent for small kernel sizes and decreases with the number of samples in the Gram matrices. Depicted is the median and the $[0.05, 0.95]$-quantiles of the MSE over 20 repetitions.

Kalman filter can be applied to nonlinear state estimation tasks as it learns the probabilistic transition and observation dynamics as linear functions on embeddings of the belief state in high-dimensional Hilbert spaces from data. In difference to existing kernel Kalman filter formulations, the KKF also provides a more general formulation that is much closer to the original Kalman filter equations and can also be applied to partially observable systems.

While the KKF can be applied to state estimation and prediction based on past observations, we extend this work by introducing the kernel forward backward smoother (KFBS) which infers the belief state from current, past, and future information. We have shown in an experimental evaluation how this additional information leads to a performance gain of the KFBS over the KKF. As kernel methods typically scale poorly with the number of data points in the kernel matrices, we have introduced a sparsification technique that leverages from the full training set while representing the embeddings only with a small subset of the data. This technique leads to significant gains of the computational efficiency while yielding similar or even slightly better results than whithout the sparsification.

We have shown that it is possible to learn the kernel Kalman rule and other kernelized inference methods also from partial observations if sliding windows of the time series provide sufficient statistics. However in future work, we want to concentrate on learning the transition dynamics in the RKHS with an expectation-maximization algorithm in case of missing information about the latent state as we think that this leads to better models of the dynamics and also improves the accuracy of the estimated variance.

References

Aronszajn N (1950) Theory of reproducing kernels. Transactions of the American Mathematical Society 68(3):337–404

Boots B, Gretton A, Gordon GJ (2013) Hilbert space embeddings of predictive state representations. In: Proceedings of the 29th International Conference on Uncertainty in Artificial Intelligence (UAI)

Chen Y, Welling M, Smola A (2012) Super-samples from kernel herding
Csató L, Opper M (2002) Sparse on-line gaussian processes. Neural computation 14(3):641–668
Drineas P, Mahoney MW (2005) On the nyström method for approximating a gram matrix for improved kernel-based learning. Journal of Machine Learning Research 6:23
Fukumizu K, Song L, Gretton A (2011) Kernel bayes' rule. In: Advances in Neural Information Processing Systems 24, Curran Associates, Inc., pp 1737–1745
Fukumizu K, Song L, Gretton A (2013) Kernel bayes' rule: Bayesian inference with positive definite kernels. Journal of Machine Learning Research 14(1):3683–3719
Gauss CF (1823) Theoria combinationis observationum erroribus minimis obnoxiae. H. Dieterich
Gebhardt GHW, Kupcsik A, Neumann G (2015) Learning subspace conditional embedding operators. Workshop on Large-Scale Kernel Learning at ICML 2015
Gebhardt GHW, Kupcsik A, Neumann G (2017) The kernel kalman rule - efficient nonparametric inference with recursive least squares. In: AAAI Conference on Artificial Intelligence
Gomez-Gonzalez S, Neumann G, Schölkopf B, Peters J (2016) Using probabilistic movement primitives for striking movements. IEEE-RAS International Conference on Humanoid Robots pp 502–508
Grünewälder S, Lever G, Baldassarre L, Patterson S, Gretton A, Pontil M (2012) Conditional mean embeddings as regressors. In: Proceedings of the 29th International Conference on Machine Learning
Hansen N (2006) The cma evolution strategy: A comparing review. In: Towards a New Evolutionary Computation. Advances on estimation of distribution algorithms, Springer, Berlin/Heidelberg, pp 75–102
Hsu D, Kakade SM, Zhang T (2012) A spectral algorithm for learning hidden markov models. Journal of Computer and System Sciences 78(5):1460–1480
Jaakkola T, Diekhans M, Haussler D (1999) Using the fisher kernel method to detect remote protein homologies. Proceedings of the International Conference on Intelligent Systems for Molecular Biology; ISMB
Jaeger H (2000) Observable operator models for discrete stochastic time series. Neural Computation 12(6):1371–1398
Julier SJ, Uhlmann JK (1997) A new extension of the kalman filter to nonlinear systems. In: Int. symp. aerospace/defense sensing, simul. and controls, vol 3, pp 3–2
Kalman RE (1960) A new approach to linear filtering and prediction problems. Journal of Fluids Engineering 82(1):35–45
Kawahara Y, Yairi T, Machida K (2007) A kernel subspace method by stochastic realization for learning nonlinear dynamical systems. Advances in Neural Information Processing Systems 19:665–672
Mccalman L, O 'callaghan S, Ramos F (2013) Multi-modal estimation with kernel embeddings for learning motion models. In: Robotics and Automation (ICRA), 2013 IEEE International Conference on, IEEE, Karlsruhe, pp 2845–2852
McElhoe BA (1966) An assessment of the navigation and course corrections for a manned flyby of mars or venus. IEEE Transactions on Aerospace and Electronic Systems AES-2
Nishiyama Y, Afsharinejad A, Naruse S, Boots B, Song L (2016) The nonparametric kernel bayes smoother. In: International Conference on Artificial Intelligence and Statistics, vol 41, pp 547–555
Rahimi A, Recht B (2007) Random features for large-scale kernel machines. Advances in Neural Information Processing Systems (1):1–8
Ralaivola L, d'Alche Buc F (2005) Time series filtering, smoothing and learning using the kernel kalman filter. Proceedings 2005 IEEE International Joint Conference on Neural Networks, 2005 3:1449–1454
Schölkopf B, Herbrich R, Smola AJ (2001) A generalized representer theorem
Shannon M, Zen H, Byrne W (2013) Autoregressive models for statistical parametric speech synthesis. IEEE Transactions on Audio, Speech and Language Processing 21(3):587–597
Simon D (2006) Optimal State Estimation. John Wiley & Sons, Inc., Hoboken, NJ, USA
Smith GL, Schmidt SF, McGee LA (1962) Application of statistical filter theory to the optimal estimation of position and velocity on board a circumlunar vehicle. National Aeronautics and Space Administration
Smola AJ, Bartlett PP (2001) Sparse greedy gaussian process regression. Advances in Neural Information Processing Systems 13 13:619–625
Smola AJ, Gretton A, Song L, Schölkopf B (2007) A hilbert space embedding for distributions. In: Proceedings of the 18th international conference on Algorithmic Learning Theory, Springer Berlin Heidelberg, Berlin, Heidelberg, Lecture Notes in Computer Science, vol 4754, pp 13–31
Snelson E, Ghahramani Z (2006) Sparse gaussian processes using pseudo-inputs. In: Advances in Neural Information Processing Systems 18, MIT Press, pp 1257–1264
Song L, Huang J, Smola A, Fukumizu K (2009) Hilbert space embeddings of conditional distributions with applications to dynamical systems. In: Proceedings of the 26th Annual International Conference on Machine Learning - ICML '09, ACM Press, New York, New York, USA, pp 1–8

Song L, Boots B, Siddiqi SM, Gordon GJ, Smola AJ (2010) Hilbert space embeddings of hidden markov models. In: Proceedings of the 27th international conference on machine learning (ICML-10), pp 991–998
Song L, Fukumizu K, Gretton A (2013) Kernel embeddings of conditional distributions: A unified kernel framework for nonparametric inference in graphical models. IEEE Signal Processing Magazine 30(4):98–111
Sorenson HW (1970) Least-squares estimation: from gauss to kalman. Spectrum, IEEE 7(7):63–68
Sun W, Capobianco R, Gordon GJ, Bagnell JA, Boots B (2016a) Learning to smooth with bidirectional predictive state inference machines. In: The Conference on Uncertainty in Artificial Intelligence (UAI 2016)
Sun W, Venkatraman A, Boots B, Bagnell JA (2016b) Learning to filter with predictive state inference machines. In: International Conference on Machine Learning, pp 1197–1205
Wan EAE, Van Der Merwe R (2000) The unscented kalman filter for nonlinear estimation. In: Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium, IEEE, pp 153–158
Williams CKI, Seeger M (2000) Using the nyström method to speed up kernel machines. In: Proceedings of the 13th International Conference on Neural Information, pp 661–667
Wojtusch J, von Stryk O (2015) Humod - a versatile and open database for the investigation, modeling and simulation of human motion dynamics on actuation level. In: IEEE-RAS International Conference on Humanoid Robots (Humanoids), IEEE
Zhu P, Chen B, Principe JC (2014) Learning nonlinear generative models of time series with a kalman filter in rkhs. IEEE Transactions on Signal Processing 62(1):141–155

A Derivations for the Subspace Conditional Operator

A.1 Derivation of the Subspace Conditional Operator

We define the subspace conditional operator $C^S_{Y|X}$ as the mapping from an embedding $\boldsymbol{\varphi}(\boldsymbol{x}) \in \mathcal{H}_X$ to the mean embedding $\boldsymbol{\mu}_{Y|\boldsymbol{x}} \in \mathcal{H}_Y$ of the conditional distribution $P(Y|\boldsymbol{x})$ conditioned on a certain variate $\boldsymbol{x}$. To obtain this subspace conditional operator, we first introduce an auxiliary conditional operator $C^{\text{aux}}_{Y|X}$ which maps from the subspace projection of the embedding $\boldsymbol{\Gamma}_x^\intercal \boldsymbol{\varphi}(\boldsymbol{x})$ to the mean map of the conditional distribution, i.e.,

$$\boldsymbol{\mu}_{Y|\boldsymbol{x}} = C^{\text{aux}}_{Y|X} \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\varphi}(\boldsymbol{x}). \tag{117}$$

We can then derive this auxiliary conditional operator by minimizing the squared error on the full data set

$$C^{\text{aux}}_{Y|X} = \arg\min_C \left\| \boldsymbol{\Phi}_y - C \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \right\|_2 \tag{118}$$

$$0 = \frac{\partial}{\partial C^{\text{aux}}_{Y|X}} \left\| \boldsymbol{\Phi}_y - C^{\text{aux}}_{Y|X} \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \right\|_2 \tag{119}$$

$$0 = -2 \left(\boldsymbol{\Phi}_y - C^{\text{aux}}_{Y|X} \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \right) \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x \tag{120}$$

$$C^{\text{aux}}_{Y|X} \boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x = \boldsymbol{\Phi}_y \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x \tag{121}$$

$$C^{\text{aux}}_{Y|X} = \boldsymbol{\Phi}_y \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x \left(\boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x + \lambda \boldsymbol{I} \right)^{-1}. \tag{122}$$

We can then substitute this result for the auxiliary conditional operator in Equation 23 and obtain the subspace conditional operator as

$$C^S_{Y|X} = C^{\text{aux}}_{Y|X} \boldsymbol{\Gamma}_x^\intercal \tag{123}$$

$$= \boldsymbol{\Phi}_y \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x \left(\boldsymbol{\Gamma}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{\Gamma}_x^\intercal \tag{124}$$

$$= \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{\Gamma}_x^\intercal, \tag{125}$$

where $\bar{\boldsymbol{K}} = \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Gamma}_x \in \mathbb{R}^{n \times m}$ is the kernel matrix of the sample feature set $\boldsymbol{\Upsilon}_x$ and its subset $\boldsymbol{\Gamma}_x$.

A.2 Comparison to Other Sparsification Approaches

The Nyström method replaces a $n \times n$ Gram matrix $\boldsymbol{K}$ with the low-rank approximation $\tilde{\boldsymbol{K}} = \boldsymbol{C}\boldsymbol{W}_k^+\boldsymbol{C}^\intercal$, where $\boldsymbol{C} \in \mathbb{R}^{n\times m}$ is a matrix of m sampled columns from $\boldsymbol{G}$ and $\boldsymbol{W}_k$ is "the best rank-k approximation to the matrix $\boldsymbol{W}$, which is a matrix whose elements consist of those elements in G in the intersection of the chosen columns and the corresponding rows" (Drineas and Mahoney 2005). Applying this technique to the conditional embedding operator applied to an embedding $\boldsymbol{\mu}_X = \boldsymbol{\Upsilon}_x \boldsymbol{m}_X$ would result in

$$\hat{C}_{Y|X}\boldsymbol{\mu}_X = \boldsymbol{\Phi}_y(\boldsymbol{C}\boldsymbol{W}_k^+\boldsymbol{C}^\intercal + \lambda \boldsymbol{I}_n)^{-1}\boldsymbol{C}\boldsymbol{W}_k^+\boldsymbol{C}^\intercal\boldsymbol{m}_X \tag{126}$$

$$= \boldsymbol{\Phi}_y\boldsymbol{C}\boldsymbol{W}_k^+(\boldsymbol{C}^\intercal\boldsymbol{C}\boldsymbol{W}_k^+ + \lambda \boldsymbol{I}_m)^{-1}\boldsymbol{C}^\intercal\boldsymbol{m}_X. \tag{127}$$

Thus, the subspace conditional operator implements the Nyström method with $\boldsymbol{C} = \tilde{\boldsymbol{K}}$ and $\boldsymbol{W} = \boldsymbol{I}$.

The idea of the random Fourier features is to compute the Fourier transform p of the kernel method k. Random samples are drawn from the distribution over frequencies p which are then used to construct a feature function $\boldsymbol{z}(\boldsymbol{x})$ which approximates the feature function intrinsic to the kernel function, i.e., $\boldsymbol{z}(\boldsymbol{x})^\intercal\boldsymbol{z}(\boldsymbol{y}) \approx k(\boldsymbol{x}, \boldsymbol{y})$. Let $\boldsymbol{Z} \in \mathbb{R}^{n\times m}$ be the feature matrix of the n data points with m random features. We could approximate the conditional embedding operator as

$$\hat{C}_{Y|X}\boldsymbol{\mu}_X = \boldsymbol{\Phi}_y(\boldsymbol{Z}\boldsymbol{Z}^\intercal + \lambda \boldsymbol{I}_n)^{-1}\boldsymbol{Z}\boldsymbol{Z}^\intercal\boldsymbol{m}_X \tag{128}$$

$$= \boldsymbol{\Phi}_y\boldsymbol{Z}(\boldsymbol{Z}^\intercal\boldsymbol{Z} + \lambda \boldsymbol{I}_m)^{-1}\boldsymbol{Z}^\intercal\boldsymbol{m}_X. \tag{129}$$

Again, it is easy to observe the similarity to our approach if we replace $\boldsymbol{Z}$ by $\tilde{\boldsymbol{K}}$.

B Derivations for the Kernel Kalman Rule and its Applications

B.1 The Residual of the Observation Operator is Unbiased

We can easily show that the residual of the observation operator is unbiased by taking the expectation

$$\mathbb{E}_Y[\boldsymbol{\zeta}_t] = \mathbb{E}_Y[\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X}\boldsymbol{\varphi}(\boldsymbol{2}x_t)] \tag{130}$$

$$= \mathbb{E}_{Y,X}[\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X}\boldsymbol{\varphi}(\boldsymbol{x}_t)] \tag{131}$$

$$= \mathbb{E}_{Y,X}[\boldsymbol{\phi}(\boldsymbol{y}_t)] - C_{Y|X}\boldsymbol{\mu}_{X,t} \tag{132}$$

$$= \mathbb{E}_X[\mathbb{E}_{Y|X}[\boldsymbol{\phi}(\boldsymbol{y}_t)]] - \mathbb{E}_X[\mathbb{E}_{Y|X}[\boldsymbol{\phi}(\boldsymbol{y}_t)]] = 0. \tag{133}$$

Here, we used the definition of the conditional embedding operator found in Song et al (2013).

B.2 Derivation of the Optimal Kalman Gain Operator

We want to find the kernel Kalman gain operator $\boldsymbol{Q}_t$ which minimizes the expected squared loss $\mathbb{E}\left[(\boldsymbol{\varepsilon}_t^+)^\intercal\boldsymbol{\varepsilon}_t^+\right]$ or equivalently the variance of the estimator. The objective for minimizing the variance can also be reformulated as minimizing the trace of the a-posteriori covariance operator $C_{XX,t}^+$ of the state $\boldsymbol{x}_t$ at time t, i.e.,

$$\min_{\boldsymbol{Q}_t}\mathbb{E}\left[(\boldsymbol{\varepsilon}_t^+)^\intercal\boldsymbol{\varepsilon}_t^+\right] = \min_{\boldsymbol{Q}_t}\operatorname{Tr}\mathbb{E}\left[\boldsymbol{\varepsilon}_t^+(\boldsymbol{\varepsilon}_t^+)^\intercal\right] \tag{134}$$

$$= \min_{\boldsymbol{Q}_t}\operatorname{Tr}\mathbb{E}\left[\left(\boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}_{X,t}^+\right)\left(\boldsymbol{\varphi}(\boldsymbol{x}_t) - \boldsymbol{\mu}_{X,t}^+\right)^\intercal\right] \tag{135}$$

$$= \min_{\boldsymbol{Q}_t}\operatorname{Tr}\, C_{XX,t}^+. \tag{136}$$

By substituting the posterior error with $\boldsymbol{\varepsilon}_t^+ = (\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X})\boldsymbol{\varepsilon}_t^- - \mathcal{Q}_t \boldsymbol{\zeta}_t$, we can now rewrite this a-posteriori covariance operator as

$$\begin{aligned}
\mathcal{C}_{XX,t}^+ &= \mathbb{E}\left[\boldsymbol{\varepsilon}_t^+ \left(\boldsymbol{\varepsilon}_t^+\right)^\intercal\right] && (137)\\
&= \mathbb{E}\left[\left(\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\boldsymbol{\varepsilon}_t^- - \mathcal{Q}_t \boldsymbol{\zeta}_t\right)\left(\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\boldsymbol{\varepsilon}_t^- - \mathcal{Q}_t \boldsymbol{\zeta}_t\right)^\intercal\right] && (138)\\
&= \left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\mathbb{E}\left[\boldsymbol{\varepsilon}_t^-(\boldsymbol{\varepsilon}_t^-)^\intercal\right]\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)^\intercal - && (139)\\
&\quad \mathcal{Q}_t \mathbb{E}\left[\boldsymbol{\zeta}_t(\boldsymbol{\varepsilon}_t^-)^\intercal\right]\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)^\intercal - \left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\mathbb{E}\left[\boldsymbol{\varepsilon}_t^-\boldsymbol{\zeta}_t^\intercal\right]\mathcal{Q}_t^\intercal + && (140)\\
&\quad \mathcal{Q}_t \mathbb{E}\left[\boldsymbol{\zeta}_t\boldsymbol{\zeta}_t^\intercal\right]\mathcal{Q}_t^\intercal && (141)
\end{aligned}$$

Since the residual of the observation operator $\boldsymbol{\zeta}_t$ is assumed to be independent from the estimation error and since we assume the a-priori estimate to be zero mean we get

$$\mathbb{E}\left[\boldsymbol{\zeta}_t(\boldsymbol{\varepsilon}_t^-)^\intercal\right] = \mathbb{E}\left[\boldsymbol{\zeta}_t\right]\mathbb{E}\left[(\boldsymbol{\varepsilon}_t^-)^\intercal\right] = \mathbb{E}\left[(\boldsymbol{\varepsilon}_t^-)^\intercal\right]\mathbb{E}\left[\boldsymbol{\zeta}_t\right] = \mathbb{E}\left[\boldsymbol{\varepsilon}_t^-\boldsymbol{\zeta}_t^\intercal\right] = 0. \tag{142}$$

With this insight the posterior covariance operator can be reformulated as

$$\mathcal{C}_{XX,t}^+ = \left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\mathcal{C}_{XX,t}^-\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)^\intercal + \mathcal{Q}_t \mathcal{R} \mathcal{Q}_t^\intercal, \tag{143}$$

where $\mathcal{R} = \mathbb{E}[\boldsymbol{\zeta}_t \boldsymbol{\zeta}_t^\intercal]$ is the covariance of the residual of the observation operator. Taking the derivative of the trace of the covariance operator and setting it to zero leads to the solution for the kernel Kalman gain operator as

$$\begin{aligned}
0 &= 2\left(\mathcal{I} - \mathcal{Q}_t \mathcal{C}_{Y|X}\right)\mathcal{C}_{XX,t}^-\left(-\mathcal{C}_{Y|X}^\intercal\right) + 2\mathcal{Q}_t \mathcal{R} && (144)\\
\mathcal{Q}_t \mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal + \mathcal{Q}_t \mathcal{R} &= \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal && (145)\\
\mathcal{Q}_t\left(\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal + \mathcal{R}\right) &= \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal && (146)\\
\mathcal{Q}_t &= \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\left(\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal + \mathcal{R}\right)^{-1}. && (147)
\end{aligned}$$

B.3 Simplifying the Update of the Covariance Operator

Following the derivations in (Simon 2006), we can find a simpler formulation for the update of the covariance operator in Equation 51. First, we substitute the kernel Kalman gain from Equation 52 using the notation $\mathcal{U} = \left(\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal + \mathcal{R}\right)$ which yields

$$\mathcal{C}_{XX,t}^+ = \left(\mathcal{I} - \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\right)\mathcal{C}_{XX,t}^-\left(\dots\right)^\intercal + \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{R}\left(\dots\right)^\intercal \tag{148}$$

By expanding both terms we obtain

$$\begin{aligned}
\mathcal{C}_{XX,t}^+ &= \mathcal{C}_{XX,t}^- - \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- - \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- + && (149)\\
&\quad \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- + && (150)\\
&\quad \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{R}\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- && (151)
\end{aligned}$$

If we now combine the second and third as well as the last two terms in this equation, we arrive at

$$\begin{aligned}
\mathcal{C}_{XX,t}^+ &= \mathcal{C}_{XX,t}^- - 2\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- + && (152)\\
&\quad \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\left(\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal + \mathcal{R}\right)\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- && (153)\\
&= \mathcal{C}_{XX,t}^- - 2\mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- + \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{U}\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- && (154)\\
&= \mathcal{C}_{XX,t}^- - \mathcal{C}_{XX,t}^-\mathcal{C}_{Y|X}^\intercal\mathcal{U}^{-1}\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- && (155)\\
&= \mathcal{C}_{XX,t}^- - \mathcal{Q}_t\mathcal{C}_{Y|X}\mathcal{C}_{XX,t}^- && (156)
\end{aligned}$$

With this update equation for the estimator covariance, which is more concise and more similar to the original equations of the Kalman filter, we can summarize the kernel Kalman rule as follows: I. compute the kernel Kalman gain operator Q_t (Equation 52), II. update the estimator of the mean embedding $\boldsymbol{\mu}^+_{X,t}$ (Equation 43), III. update the estimator of the covariance embedding $C^+_{XX,t}$ (Equation 53).

B.4 Derivations for the Sample-based Kernel Kalman Rule

The Kalman gain operator can be rewritten with the sample estimators of mean embedding, covariance embedding and conditional embedding operator as

$$Q_t = \boldsymbol{\Upsilon}_{\boldsymbol{x}} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal \boldsymbol{\Phi}_{\boldsymbol{y}}^\intercal \left(\boldsymbol{\Phi}_{\boldsymbol{y}} \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal \boldsymbol{\Phi}_{\boldsymbol{y}}^\intercal + \kappa \mathcal{I} \right)^{-1}, \tag{157}$$

However, this formulation still has the inversion of a potentially infinite dimensional operator. To this end, we can apply the matrix identity $\boldsymbol{A}(\boldsymbol{B}\boldsymbol{A} + \boldsymbol{I})^{-1} = (\boldsymbol{A}\boldsymbol{B} + \boldsymbol{I})^{-1}\boldsymbol{A}$ to the kernel Kalman gain matrix as follows

$$\begin{aligned} Q_t &= \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{O}^\intercal \underbrace{\boldsymbol{\Phi}_y^\intercal}_{\text{A}} \Big[\underbrace{\boldsymbol{\Phi}_y \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal}_{\text{B}} \underbrace{\boldsymbol{\Phi}_y^\intercal}_{\text{A}} + \kappa \mathcal{I} \Big]^{-1} && (158) \\ &= \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\underbrace{\boldsymbol{\Phi}_y^\intercal}_{\text{A}} \underbrace{\boldsymbol{\Phi}_y \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal}_{\text{B}} + \kappa \boldsymbol{I})^{-1} \underbrace{\boldsymbol{\Phi}_y^\intercal}_{\text{A}} && (159) \\ &= \boldsymbol{\Upsilon}_x \underbrace{\boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\boldsymbol{G} \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal + \kappa \boldsymbol{I})^{-1}}_{\boldsymbol{Q}_t} \boldsymbol{\Phi}_y^\intercal, && (160) \end{aligned}$$

where we defined $\boldsymbol{Q}_t = \boldsymbol{S}_t^- \boldsymbol{O}^\intercal (\boldsymbol{G} \boldsymbol{O} \boldsymbol{S}_t^- \boldsymbol{O}^\intercal + \kappa \boldsymbol{I})^{-1} \in \mathbb{R}^{n \times n}$ with the Gram matrix of the observations $\boldsymbol{G} = \boldsymbol{\Phi}_y^\intercal \boldsymbol{\Phi}_y$. Based on this formulation of the kernel Kalman gain operator, we can now derive finite vector/matrix formulations for the update equations of the estimator of the mean embedding (Equation 43) and the estimator of the covariance operator (Equation 53). For the estimator of the mean embedding, we obtain

$$\begin{aligned} \boldsymbol{\mu}^+_{X,t} &= \boldsymbol{\mu}^-_{X,t} + Q_t \left(\boldsymbol{\phi}(\boldsymbol{y}_t) - C_{Y|X} \boldsymbol{\mu}^-_{X,t} \right) && (161) \\ \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^+ &= \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^- + \boldsymbol{\Upsilon}_x \boldsymbol{Q}_t \boldsymbol{\Phi}_y^\intercal \left(\boldsymbol{\phi}(\boldsymbol{y}_t) - \boldsymbol{\Phi}_y \left(\boldsymbol{K} + \lambda \boldsymbol{I}_m \right)^{-1} \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^- \right), && (162) \\ &= \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^- + \boldsymbol{\Upsilon}_x \boldsymbol{Q}_t \left(\boldsymbol{\Phi}_y^\intercal \boldsymbol{\phi}(\boldsymbol{y}_t) - \boldsymbol{\Phi}_y^\intercal \boldsymbol{\Phi}_y \left(\boldsymbol{K} + \lambda \boldsymbol{I}_m \right)^{-1} \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{m}_t^- \right), && (163) \\ \boldsymbol{m}_t^+ &= \boldsymbol{m}_t^- + \boldsymbol{Q}_t \left(\boldsymbol{g}_{:\boldsymbol{y}_t} - \boldsymbol{G} \boldsymbol{O} \boldsymbol{m}_t^- \right), && (164) \end{aligned}$$

where $\boldsymbol{g}_{:\boldsymbol{y}_t} = \boldsymbol{\Phi}_y^\intercal \boldsymbol{\phi}(\boldsymbol{y}_t)$ is the embedding of the measurement at time t. And the estimator for the covariance operator gets

$$\begin{aligned} \hat{C}^+_{XX,t} &= \hat{C}^-_{XX,t} - \boldsymbol{\Upsilon}_x \boldsymbol{Q}_t \boldsymbol{\Phi}_y^\intercal C_{Y|X} \hat{C}^-_{XX,t} && (165) \\ \boldsymbol{\Upsilon}_x \boldsymbol{S}_t \boldsymbol{\Upsilon}_x^\intercal &= \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{\Upsilon}_x^\intercal - \boldsymbol{\Upsilon}_x \boldsymbol{Q}_t \boldsymbol{\Phi}_y^\intercal \boldsymbol{\Phi}_y \left(\boldsymbol{K} + \lambda \boldsymbol{I}_m \right)^{-1} \boldsymbol{\Upsilon}_x^\intercal \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{\Upsilon}_x^\intercal && (166) \\ &= \boldsymbol{\Upsilon}_x \boldsymbol{S}_t^- \boldsymbol{\Upsilon}_x^\intercal - \boldsymbol{\Upsilon}_x \boldsymbol{Q}_t \boldsymbol{G} \left(\boldsymbol{K} + \lambda \boldsymbol{I}_m \right)^{-1} \boldsymbol{K} \boldsymbol{S}_t^- \boldsymbol{\Upsilon}_x^\intercal && (167) \\ \boldsymbol{S}_t &= \boldsymbol{S}_t^- - \boldsymbol{Q}_t \boldsymbol{G} \boldsymbol{O} \boldsymbol{S}_t^-. && (168) \end{aligned}$$

B.5 Derivation of the Subspace Kernel Kalman Gain Operator/Matrix

To derive the gain operator of the subspace kernel Kalman rule, we apply the subspace conditional operator

$$C^S_{Y|X} = \boldsymbol{\Phi}_y \bar{\boldsymbol{K}} \left(\bar{\boldsymbol{K}}^\intercal \bar{\boldsymbol{K}} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{\Gamma}_x^\intercal, \tag{169}$$

to the kernel Kalman gain from Equation 52 and obtain

$$Q_t^S = C_{XX,t}^- \left(C_{Y|X}^S\right)^\intercal \left(C_{Y|X}^S C_{XX,t}^- \left(C_{Y|X}^S\right)^\intercal + \kappa \mathcal{I}\right)^{-1} \tag{170}$$

$$= \Upsilon_x S_t^- \bar{K}(O^S)^\intercal \underbrace{\bar{K}^\intercal \Phi_y^\intercal}_{A} \Big(\underbrace{\Phi_y \bar{K} O^S \bar{K}^\intercal S_t^- \bar{K}(O^S)^\intercal}_{B} \underbrace{\bar{K}^\intercal \Phi_y^\intercal}_{A} + \kappa \mathcal{I}\Big)^{-1}. \tag{171}$$

Here, we denote $O^S := \left(\bar{K}^\intercal \bar{K} + \lambda I\right)^{-1}$. To obtain a finite dimensional matrix in the inverse, we apply again the matrix identity $A(BA + I)^{-1} = (AB + I)^{-1}A$ and arrive at

$$Q_t^S = \Upsilon_x S_t^- \bar{K}(O^S)^\intercal \Big(\underbrace{\bar{K}^\intercal \Phi_y^\intercal}_{A} \underbrace{\Phi_y \bar{K} O^S \bar{K}^\intercal S_t^- \bar{K}(O^S)^\intercal}_{B} + \kappa I\Big)^{-1} \underbrace{\bar{K}^\intercal \Phi_y^\intercal}_{A} \tag{172}$$

$$= \Upsilon_x S_t^- \bar{K}(O^S)^\intercal \left(\bar{K}^\intercal G \bar{K} O^S \bar{K}^\intercal S_t^- \bar{K}(O^S)^\intercal + \kappa I\right)^{-1} \bar{K}^\intercal \Phi_y^\intercal. \tag{173}$$

Using the projecting the subspace kernel Kalman gain into the subspace spanned by the features $\Gamma_x^\intercal$ leads to

$$\Gamma_x^\intercal Q_t^S = \Gamma_x^\intercal \Upsilon_x S_t^- \bar{K}(O^S)^\intercal \left(\bar{K}^\intercal G \bar{K} O^S \bar{K}^\intercal S_t^- \bar{K}(O^S)^\intercal + \kappa I\right)^{-1} \bar{K}^\intercal \Phi_y^\intercal \tag{174}$$

$$= \underbrace{P_t^- (O^S)^\intercal \left(\bar{K}^\intercal G \bar{K} O^S P_t^- (O^S)^\intercal + \kappa I\right)^{-1} \bar{K}^\intercal}_{Q_t^S} \Phi_y^\intercal, \tag{175}$$

where we define the subspace kernel Kalman gain matrix Q_t^S. Using this representation, we can obtain the update equations for the subspace projections of mean embedding as

$$\Gamma_x^\intercal \mu_{X,t}^+ = \Gamma_x^\intercal \left(\mu_{X,t}^- + Q_t^S \left(\phi(y_t) - C_{Y|X}^S \mu_{X,t}^-\right)\right) \tag{176}$$

$$n_{X,t}^+ = n_{X,t}^- + Q_t^S \left(g_{:y_t} - G \bar{K} O^S n_t^-\right), \tag{177}$$

and similarly for the covariance operator as

$$\Gamma_x^\intercal \hat{C}_{XX,t}^+ \Gamma_x = \Gamma_x^\intercal \left(\hat{C}_{XX,t}^- - Q_t^S C_{Y|X}^S \hat{C}_{XX,t}^-\right) \Gamma_x \tag{178}$$

$$P_t^+ = P_t^- - Q_t^S \Phi_y^\intercal \Phi_y \bar{K} O^S \Gamma_x^\intercal \hat{C}_{XX,t}^- \Gamma_x \tag{179}$$

$$= P_t^- - Q_t^S G \bar{K} O^S P_t^- \tag{180}$$